Analisis Algoritma Kruskal dan Prim pada Masalah *Minimum Spanning Tree*: Kompleksitas dan Eksperimen Empiris dengan Pembanding Borůvka

Muhammad Alif Qadri (D082261018)
Magister Teknik Informatika
Universitas Hasanuddin
Makassar, Indonesia
alifqadry@gmail.com

Abstrak: Pada graf berbobot, *minimum spanning tree* (MST) adalah himpunan sisi yang menghubungkan semua simpul dengan total bobot paling kecil. Laporan ini membahas dua algoritma pencarinya, Kruskal dan Prim, secara teoretis dan empiris, dengan Borůvka sebagai pembanding ketiga. Bagian teori berisi contoh manual, *pseudocode*, dan bukti kebenaran Kruskal dan Prim. Kompleksitas waktu dan ruang diturunkan untuk kasus terbaik, rata-rata, dan terburuk, dibedakan menurut kepadatan graf. Bila memakai *binary heap* tanpa operasi *decrease-key*, Kruskal dan Prim sama-sama berwaktu $O(m \log n)$. Borůvka pada kode ini berwaktu $O(m \log^2 n)$. Ketiganya diimplementasikan dalam Python. Kebenarannya diperiksa dengan empat kasus uji kecil, perbandingan terhadap `networkx` pada 300 graf acak, dan studi kasus jaringan kabel lima gedung yang biayanya hanya ilustrasi. Waktu eksekusi diukur di Google Colab pada tiga skenario graf: jarang, padat, dan jarang dengan bobot kembar. Tiap skenario memakai lima ukuran dan lima ulangan. Kruskal memiliki rata-rata waktu terendah pada graf jarang, sedangkan Prim terendah pada graf padat dengan syarat daftar ketetanggaan sudah tersedia. Selisih keduanya sering masih berada di dalam simpangan baku. Borůvka paling lambat, yaitu 3,34 sampai 4,72 kali waktu Kruskal pada ukuran terbesar. Waktu Kruskal dan Prim juga tumbuh lebih cepat daripada $m \log n$ pada rentang yang diukur, tetapi penyebabnya tidak diuji. Semua temuan ini terbatas pada implementasi Python dan rentang ukuran yang dipakai.

Kata kunci: minimum spanning tree, algoritma Kruskal, algoritma Prim, algoritma Borůvka, kompleksitas waktu, eksperimen empiris

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Dalam teori graf dan optimasi kombinatorial, *minimum spanning tree* (MST) merupakan salah satu masalah paling mendasar. Diberikan graf tak berarah, terhubung, dan berbobot, MST adalah himpunan sisi yang menghubungkan seluruh simpul tanpa membentuk siklus dengan total bobot sekecil mungkin. Masalah ini dibahas dalam bab tersendiri pada buku teks algoritma, misalnya Bab 11 pada Sanders *et al.* [1].

Kegunaan MST sudah tampak sejak algoritmanya dirumuskan. Prim [2] menyebut bahwa masalah ini muncul dalam perencanaan jaringan komunikasi, distribusi, dan transportasi berskala besar. Hingga kini, MST menjadi komponen dasar pada analisis jaringan, dan juga dipakai pada desain *chip*, pelacakan mata (*eye tracking*), perencanaan rute, serta diagnostik medis seperti pengenalan tumor [3]. Di luar bidang tersebut, MST menjadi representasi data pada banyak tugas pengenalan pola dan relatif cepat dihitung [4]. Penerapannya mencakup *clustering*, segmentasi citra, dan desain jaringan [5], serta deteksi anomali dan optimasi jaringan pada data berukuran besar [6].

Dua solusi klasik untuk MST adalah algoritma Kruskal dan algoritma Prim. Kruskal [7] mengusulkan algoritma untuk menemukan MST pada suatu graf, dan setahun kemudian Prim [2] menerbitkan algoritma yang kini membawa namanya. Algoritma ini juga dikenal sebagai Jarník-Prim [8], dan selanjutnya laporan ini menyebutnya Prim. Keduanya tergolong algoritma *greedy* [3], tetapi cara kerjanya berbeda. Kruskal menumbuhkan sebuah hutan dengan memindai sisi, sedangkan Prim menumbuhkan satu pohon mulai dari simpul sembarang [8].

Meskipun keduanya menghasilkan MST, biaya komputasinya tidak sama pada setiap graf. Misalkan graf memiliki n simpul dan m sisi. Prim dengan antrean prioritas yang efisien berjalan dalam waktu $O(m + n \log n)$, sedangkan Kruskal berjalan dalam waktu $O((m + n) \log m)$ [8]. Batas untuk Prim dicapai, misalnya, dengan *Fibonacci heap* [1, Bag. 11.2]. Implementasi dalam laporan ini memakai *binary heap*, sehingga batas yang berlaku untuk Prim adalah $O(m \log n)$, dan penurunannya dibahas di Bab 3. Karena itu, kepadatan graf dan pilihan struktur data menjadi penentu. Percobaan Osipov *et al.* [8] dengan implementasi C++ pada graf acak berbobot acak menunjukkan bahwa Kruskal bekerja baik hingga sekitar $8n$ sisi, sedangkan Prim lebih unggul pada graf yang cukup padat untuk ukuran graf yang diuji. Hasil ini belum tentu berlaku untuk implementasi dalam laporan ini.

Batas asimtotik saja belum cukup untuk menentukan algoritma yang lebih baik dalam praktik, sebab waktu eksekusi nyata juga dipengaruhi detail implementasi. Pada ECL-MST, implementasi GPU yang memadukan algoritma Kruskal dan Borůvka, delapan optimasi yang dievaluasi membuat kodenya lebih dari 8 kali lebih cepat daripada versi dasarnya [3]. Karena itu, analisis teoretis perlu dilengkapi pengukuran empiris yang dibandingkan langsung dengan kurva kompleksitasnya.

Berdasarkan uraian tersebut, laporan ini menganalisis algoritma Kruskal dan Prim secara formal, mencakup bukti kebenaran serta penurunan kompleksitas waktu dan ruang pada kasus terbaik, rata-rata, dan terburuk. Analisis itu kemudian diuji melalui eksperimen pada graf jarang dan padat dengan ukuran input bervariasi, lalu hasilnya dibandingkan dengan satu algoritma pembanding ketiga, yaitu algoritma Borůvka [9].

1.2 Definisi Formal Masalah

Secara sederhana, masalah MST adalah mencari cara berbobot total terkecil untuk menghubungkan seluruh simpul pada suatu graf. Titik yang akan dihubungkan berperan sebagai simpul, penghubung yang mungkin dipasang di

antara dua titik sebagai sisi, dan ukuran harga penghubung itu sebagai bobot.

Secara formal, diberikan graf tak berarah dan terhubung $G = (V, E)$, dengan V himpunan simpul, E himpunan sisi, dan $w: E \to \mathbb{R}$ fungsi yang memberi setiap sisi e sebuah bobot bilangan real $w(e)$. *Spanning tree* dari G adalah himpunan sisi $T \subseteq E$ yang menghubungkan seluruh simpul tanpa membentuk siklus, dan *minimum spanning tree* adalah *spanning tree* dengan total bobot terkecil [1, Bab 11], [9]. Total bobot sebuah *spanning tree* adalah

$$w(T) = \sum_{e \in T} w(e) \quad (1)$$

Minimum spanning tree adalah *spanning tree* T^* yang, untuk setiap *spanning tree* T dari G, memenuhi

$$w(T^*) \leq w(T) \quad (2)$$

Dengan notasi tersebut, masalah MST dirumuskan sebagai berikut.

Masukan: graf tak berarah dan terhubung $G = (V, E)$ dengan fungsi bobot $w: E \to \mathbb{R}$.

Keluaran: sebuah *spanning tree* T^* dari G dengan $w(T^*)$ minimum.

Ukuran input dinyatakan dengan $n = |V|$ dan $m = |E|$. Setiap *spanning tree* memiliki tepat $n - 1$ sisi, dan $m \leq n(n-1)/2$ untuk graf tanpa sisi ganda dan tanpa *loop*. Selain itu, $m \geq n - 1$ karena G terhubung. Graf disebut jarang jika $m = \Theta(n)$, padat jika $m = \Theta(n^2)$, dan berkepadatan menengah jika $m = \Theta(n \log n)$. Ketiga tingkat kepadatan ini dipakai pada subbab 2.3 untuk mendefinisikan kasus terbaik, rata-rata, dan terburuk.

Karena G terhubung, G memiliki sedikitnya satu *spanning tree*, sehingga MST selalu ada. Bobot sisi boleh bernilai sama. Dalam kasus ini MST dapat lebih dari satu, tetapi seluruhnya memiliki total bobot yang sama.

1.3 Tujuan

Berdasarkan latar belakang dan definisi masalah di atas, tujuan kajian ini adalah sebagai berikut.

1. Menjelaskan prinsip kerja algoritma Kruskal dan Prim beserta *pseudocode* dan contoh perhitungan manualnya sebagai penerapan paradigma *greedy* pada masalah MST.
2. Membuktikan bahwa algoritma Kruskal dan Prim selalu menghasilkan *minimum spanning tree* pada graf tak berarah, terhubung, dan berbobot.
3. Menurunkan kompleksitas waktu dan ruang kedua algoritma secara formal pada kasus terbaik, rata-rata, dan terburuk.
4. Mengimplementasikan algoritma Kruskal dan Prim serta algoritma Borůvka sebagai pembanding, memvalidasi kebenarannya melalui kasus uji kecil yang hasilnya dapat diperiksa secara manual, serta menerapkannya pada satu studi kasus jaringan kabel antar gedung di Fakultas Teknik Universitas Hasanuddin.
5. Mengukur waktu eksekusi ketiga algoritma pada graf jarang dan padat dengan berbagai ukuran input, lalu membandingkan hasilnya satu sama lain dan dengan kurva kompleksitas teoretis.
6. Mengevaluasi kelebihan, keterbatasan, kesesuaian, dan alternatif algoritma berdasarkan hasil analisis dan eksperimen.

1.4 Batasan Penelitian

Ruang lingkup kajian ini dibatasi sebagai berikut.

1. Masalah yang dikaji adalah MST pada graf tak berarah, terhubung, dan berbobot bilangan real, tanpa sisi ganda dan tanpa *loop*. Graf tak terhubung, yang menghasilkan *minimum spanning forest*, tidak dibahas.
2. Algoritma yang dianalisis secara formal dan diimplementasikan adalah Kruskal dengan struktur data *disjoint set* dan Prim dengan antrean prioritas berbasis *binary heap* yang menyimpan sisi kandidat. Borůvka, satu-satunya pembanding yang ditulis sendiri, hanya diimplementasikan dan dibandingkan secara empiris. Batas kompleksitasnya diturunkan untuk kode yang ditulis dan dibandingkan dengan batas pada literatur, tanpa bukti kebenaran formal. Varian lain seperti Filter-Kruskal, serta versi paralel dan dinamis tidak dibahas.
3. Implementasi ditulis dalam Python dan berjalan secara sekuensial. Logika inti Kruskal, Prim, dan Borůvka serta *disjoint set* ditulis sendiri, sedangkan fungsi pengurutan bawaan dan modul `heapq` hanya dipakai sebagai alat bantu. Pustaka yang langsung menghasilkan MST hanya dipakai untuk verifikasi hasil.
4. Graf masukan untuk Kruskal dan Borůvka direpresentasikan sebagai daftar sisi, sedangkan untuk Prim sebagai daftar ketetanggaan.
5. Data uji pengukuran waktu berupa graf acak sintetis yang dibangkitkan dengan *seed* tetap dan dijamin terhubung, pada kondisi jarang, padat, dan jarang berbobot kembar dengan minimal lima ukuran input per kondisi. Rentang ukuran ditetapkan di Bab 4 dengan mempertimbangkan bahwa jumlah sisi graf padat tumbuh sebesar $\Theta(n^2)$. Studi kasus berupa satu graf kecil jaringan kabel antar lima gedung dengan data tetap, sehingga tidak memakai *seed* dan tidak dipakai untuk mengukur waktu.
6. Pengukuran empiris terbatas pada waktu eksekusi, diulang minimal lima kali per percobaan pada graf yang sama, dan dilaporkan sebagai rata-rata dan simpangan baku pada satu lingkungan perangkat keras dan perangkat lunak. Kompleksitas ruang hanya dianalisis secara teoretis.
7. Ketiga algoritma memakai aturan pemutus seri yang sama untuk sisi berbobot sama, yaitu membandingkan bobot, lalu nomor simpul ujung terkecil, lalu nomor simpul ujung terbesar, dengan simpul diberi nomor bulat tetap. Dengan aturan ini seluruh sisi terurut total, sehingga keluaran ketiga algoritma dapat dibandingkan.

BAB 2 LANDASAN TEORI

2.1 Paradigma Desain Greedy

2.1.1 Sifat Greedy-Choice dan Optimal Substructure

Algoritma *greedy* adalah strategi penyelesaian masalah yang memeriksa unsur-unsur masalah dalam suatu urutan, dan keputusan tentang suatu unsur diambil pada saat unsur itu diperiksa. Keputusan yang sudah diambil tidak pernah dibatalkan [1, Bab 12]. Pada umumnya strategi ini hanya menghasilkan solusi yang suboptimal, tetapi pada beberapa masalah, termasuk MST dan lintasan terpendek dengan bobot tak negatif, hasilnya optimal [1, Bab 12]. Masalah yang dapat diselesaikan secara *greedy* memiliki dua sifat utama, yaitu *greedy-choice* dan *optimal substructure* [10].

Sifat *greedy-choice* berarti pilihan yang optimal secara lokal menuntun ke solusi yang optimal secara global. Kebenarannya harus dibuktikan, pada MST dengan argumen *cut and paste*, yaitu menukar sebagian solusi optimal dengan pilihan *greedy* tanpa menambah biaya [10], [1, Bag. 11.1].

Suatu masalah memiliki *optimal substructure* jika solusi optimalnya tersusun dari solusi optimal submasalahnya [10].

Pada MST, *optimal substructure* ditunjukkan dengan kontraksi sisi. Jika sisi e dari suatu MST dikontraksi dengan menggabungkan kedua simpul ujungnya, dan T' adalah MST dari graf hasil kontraksi, maka T' ditambah e adalah MST dari graf semula. Untuk *greedy-choice*, pada setiap potongan (*cut*) graf, sisi berbobot terkecil yang melintasi potongan termasuk dalam suatu MST [10], [1, Lemma 11.1]. Sifat ini menjadi dasar algoritma Prim dan Kruskal, yang menjaga agar himpunan sisi terkumpul selalu menjadi bagian dari suatu MST. Bukti kebenaran keduanya dibahas di Bab 3.

2.1.2 Penerapan pada MST: Strategi Kruskal (Hutan) dan Prim (Satu Pohon)

Kruskal dan Prim adalah kasus khusus dari algoritma generik yang bertumpu pada *cut property*. Algoritma generik dimulai dari himpunan sisi kosong A. Selama A belum menjadi *spanning tree*, dipilih sebuah potongan yang tidak dilintasi sisi dari A, lalu sisi berbobot minimum pada potongan itu ditambahkan. Pilihan potongan yang berbeda menghasilkan algoritma yang berbeda [1, Bag. 11.1]. Sisi yang dapat ditambahkan ke A tanpa melanggar syarat bahwa A termuat dalam suatu MST disebut *safe edge* dalam laporan ini. Selama algoritma berjalan, (V, A) selalu berupa hutan, dan setiap sisi yang ditambahkan menghubungkan dua komponen yang berbeda sehingga tidak membentuk siklus. Jika C adalah salah satu komponen dan (u, v) adalah sisi berbobot minimum di antara sisi yang meninggalkan C, maka (u, v) adalah *safe edge* menurut Teorema 2.1 pada potongan $(C, V \setminus C)$. Akibat inilah yang dipakai kedua algoritma untuk memilih sisi.

Pada algoritma Kruskal, A berupa hutan. Algoritma mengurutkan seluruh sisi menurut bobot tak menurun, lalu memeriksanya satu per satu. Jika kedua ujung sisi sudah berada di pohon yang sama, sisi dibuang karena penambahannya membentuk siklus. Jika tidak, sisi ditambahkan ke A sebagai *light edge* pada potongan yang menghormati A [1, Bag. 11.3], [7]. Argumen lengkapnya ada pada 3.3.2. Pemeriksaan keanggotaan pohon memakai struktur data *disjoint set* (*union-find*) dengan operasi *find* dan *union* [1, Bag. 11.4]. Algoritma ini bekerja pada graf yang tersedia sebagai urutan sisi [1, Bag. 11.3].

Pada algoritma Prim, A selalu membentuk satu pohon yang tumbuh dari simpul awal sembarang. Misalkan S adalah himpunan simpul yang sudah masuk pohon. Pada setiap iterasi, sisi berbobot minimum yang meninggalkan S, yaitu sisi dengan tepat satu ujung di S, ditambahkan ke pohon [1, Bag. 11.2], [2]. Cara kerja ini mirip algoritma Dijkstra, dengan perbedaan bahwa antrean prioritas menyimpan biaya sisi, bukan panjang lintasan [1, Bag. 11.2]. Rumusan tersebut memerlukan operasi *decrease-key*. Implementasi dalam laporan ini tidak memakainya dan menyimpan sisi, bukan simpul, di dalam antrean. Alasan dan akibatnya pada kompleksitas dibahas pada 3.2.3 dan 3.4.3. Perbedaan strategi dan struktur data inilah yang menentukan biaya komputasi masing-masing algoritma.

2.2 Landasan Matematis MST

2.2.1 Cut, Light Edge, dan Cut Property

Misalkan $G = (V, E)$ adalah graf tak berarah dan terhubung dengan fungsi bobot w seperti pada 1.2. Potongan (*cut*) dari G adalah pembagian himpunan simpul V menjadi dua himpunan tak kosong $(S, V \setminus S)$. Sebuah sisi $(u, v) \in E$ melintasi potongan jika salah satu ujungnya berada di S dan ujung lainnya di $V \setminus S$. Himpunan sisi yang melintasi potongan dinotasikan $E(S, V \setminus S)$ [1, Bag. 11.1]. Sebuah sisi $e \in E(S, V \setminus S)$ disebut *light edge* dalam laporan ini jika bobotnya minimum di antara sisi yang melintasi potongan tersebut [1, Bag. 11.1], yaitu

$$w(e) = \min\{w(f) : f \in E(S, V \setminus S)\} \tag{3}$$

Jika beberapa sisi memiliki bobot minimum yang sama, potongan memiliki lebih dari satu *light edge*. Suatu potongan disebut menghormati (*respects*) himpunan sisi $A \subseteq E$ jika tidak ada sisi di A yang melintasi potongan tersebut, sesuai syarat pada Lemma 11.1 dalam Sanders *et al.* [1]. Sifat yang mengaitkan potongan dengan MST disebut *cut property* [1, Bag. 11.1] atau *cut rule* [9, Bag. 8].

Teorema 2.1 (*Cut Property*). Misalkan $G = (V, E)$ graf tak berarah, terhubung, dan berbobot real. Misalkan $A \subseteq E$ termuat dalam suatu MST dari G, $(S, V \setminus S)$ potongan yang menghormati A, dan (u, v) sebuah *light edge* yang melintasi potongan tersebut. Maka $A \cup \{(u, v)\}$ juga termuat dalam suatu MST dari G, atau dengan kata lain (u, v) adalah *safe edge* untuk A [1, Lemma 11.1].

Rumusan ini memakai bobot real, lebih luas daripada asumsi bobot positif pada buku tersebut. Bukti penukarannya tidak memakai tanda bobot, dan masalah MST dapat diselesaikan untuk fungsi bobot apa pun, termasuk bobot negatif [9, Bag. 8]. Bukti lengkapnya ada pada 3.3.1. Pada bobot yang tidak seluruhnya berbeda, teorema hanya menjamin bahwa *light edge* termuat dalam suatu MST. Salah satu cara menanganinya adalah aturan pemutus seri yang konsisten [9, Bag. 4, catatan ⟨1⟩, hlm. 23], [5, Bag. II-C].

2.2.2 Cycle Property

Jika *cut property* menunjukkan sisi yang dapat dimasukkan ke dalam MST, *cycle property* menunjukkan sisi yang dapat dikeluarkan.

Teorema 2.2 (*Cycle Property*). Misalkan C adalah sebuah siklus di graf terhubung berbobot real $G = (V, E)$ dan e adalah sisi pada C dengan bobot maksimum di antara semua sisi C. Maka setiap MST dari $G' = (V, E \setminus \{e\})$ juga merupakan MST dari G [1, Lemma 11.2].

Buktinya memakai argumen penukaran yang serupa dengan *cut property* dan tidak diulang di Bab 3, karena bukti kebenaran Kruskal dan Prim hanya memakai *cut property*. Pada Kruskal, sisi yang menutup siklus dibuang sesuai *cycle property* dan sisi yang menghubungkan dua komponen ditambahkan sesuai *cut property* [1, Bag. 11.3].

2.2.3 Keunikan MST pada Bobot Berbeda dan Bobot Sama

Sebuah graf dapat memiliki lebih dari satu MST. Segitiga yang ketiga sisinya berbobot 1, misalnya, memiliki tiga MST berbobot 2. Seluruh MST dari suatu graf memiliki total bobot yang sama, yaitu bobot minimum menurut (2).

Teorema 2.3 (Keunikan MST). Jika semua bobot sisi pada graf terhubung G berbeda, maka G memiliki tepat satu MST [9, Bag. 7], [1, Latihan 11.5].

Teorema ini dibuktikan pada 3.3.4 dengan syarat yang lebih umum, yaitu setiap potongan hanya memiliki satu *light edge*. Kebalikannya tidak berlaku. Pohon dengan bobot berulang tetap memiliki MST yang unik karena satu-satunya *spanning tree* dari pohon adalah pohon itu sendiri.

Jika bobot tidak berbeda, keluaran Kruskal dapat berbeda untuk graf yang sama bergantung pada urutan sisi berbobot sama saat pengurutan. Ambiguitas ini dihilangkan dengan aturan pemutus seri yang konsisten [9, Bag. 4, catatan ⟨1⟩, hlm. 23], misalnya dengan label simpul [5, Bag. II-C].

Aturan yang dipakai laporan ini membandingkan bobot terlebih dahulu, lalu simpul ujung bernomor terkecil, lalu simpul ujung bernomor terbesar. Karena graf tidak memiliki sisi ganda, setiap sisi memiliki tripel (w, u, v) dengan $u < v$ yang berbeda, sehingga seluruh sisi terurut total dan keluaran Kruskal dan Prim menjadi tunggal (dibuktikan pada 3.3.4). Tanpa aturan yang sama, dua algoritma yang benar dapat menghasilkan himpunan sisi berbeda dengan total bobot sama.

2.3 Notasi Asimtotik dan Definisi Kasus Analisis

Kompleksitas algoritma dinyatakan dengan notasi asimtotik O, Ω, dan Θ, yang mengabaikan konstanta pengali dan suku berorde rendah karena yang diperhatikan adalah laju pertumbuhan untuk masukan besar. Notasi O menyatakan batas atas, Ω batas bawah, dan Θ keduanya [1, Bag. 2.1]. Ukuran masukan graf dinyatakan dengan $n = |V|$ dan $m = |E|$ [1, Bag. 2.12], sehingga notasi O dipakai untuk fungsi dua peubah.

Misalkan I_n adalah himpunan hingga masukan berukuran n dan $T(i)$ adalah waktu berjalan pada masukan i. Kasus terburuk, terbaik, dan rata-rata didefinisikan sebagai [1, Bag. 2.1]

$$\begin{aligned} T_{\text{terburuk}}(n) &= \max\{T(i) : i \in I_n\}, \\ T_{\text{terbaik}}(n) &= \min\{T(i) : i \in I_n\}, \\ T_{\text{rata-rata}}(n) &= \frac{1}{|I_n|}\sum_{i \in I_n} T(i) \end{aligned} \tag{4}$$

Kasus terburuk memberikan jaminan kinerja yang paling kuat, sedangkan kasus rata-rata memerlukan model keacakan [1, Bag. 2.1 dan 2.9]. Pada MST, laporan ini membedakan ketiga kasus berdasarkan kepadatan graf, yaitu berdasarkan jumlah sisi m untuk n tertentu dengan batas $n - 1 \le m \le n(n-1)/2$ (1.2). Pembagian ini adalah definisi yang dipilih dalam laporan ini dan ditunjukkan pada Tabel 2.1.

TABEL 2.1. DEFINISI KASUS ANALISIS BERDASARKAN KEPADATAN GRAF

Kasus	Kepadatan graf (untuk n tertentu)	Jumlah sisi
Terbaik	Graf jarang minimal (berbentuk pohon)	$m = n - 1$
Rata-rata	Kepadatan menengah sebagai wakil	$m = \Theta(n \log n)$
Terburuk	Graf padat	$m = \Theta(n^2)$

Batas waktu tiap kasus diperoleh dengan mensubstitusikan nilai m. Klasifikasi ini tidak sama dengan nilai minimum, rata-rata, dan maksimum waktu berjalan atas seluruh masukan berukuran n pada (4), terutama untuk kasus rata-rata, yang di sini bukan nilai harapan probabilistik melainkan kepadatan menengah yang dipilih sebagai wakil. Kasus terbaik termasuk graf jarang $\left(m = \Theta(n)\right)$ dan kasus terburuk adalah graf padat, sehingga keduanya dapat diukur pada eksperimen. Hasil analisis di Bab 3 dinyatakan dengan notasi O, yaitu batas atas untuk urutan bobot sisi apa pun pada kepadatan yang bersangkutan. Pernyataan bahwa Θ berlaku memerlukan argumen batas bawah tersendiri.

2.4 Algoritma Borůvka sebagai Pembanding

Algoritma Borůvka dipilih sebagai pembanding karena strateginya berbeda dari Kruskal dan Prim, tetapi tetap bertumpu pada sifat *greedy* yang sama. Borůvka merumuskan solusi pertama masalah MST pada tahun 1926, dan kedua makalah aslinya diterjemahkan serta dikaji oleh Nešetřil *et al.* [9]. Algoritma ini bekerja dalam putaran. Mula-mula setiap simpul adalah pohon tersendiri. Pada setiap putaran, setiap pohon memilih sisi berbobot minimum yang keluar dari pohon itu, semua sisi terpilih ditambahkan ke hutan, dan pohon-pohon yang terhubung digabung, sampai tersisa satu pohon [9, Bag. 7]. Jumlah pohon berkurang paling sedikit setengahnya pada setiap putaran, sehingga putaran paling banyak $\log n$, dan setiap putaran dapat dilakukan dalam waktu linear terhadap m, sehingga totalnya $O(m \log n)$ [9, Bag. 8], [1, Bag. 11.6]. Penurunan batas untuk kode yang ditulis disajikan singkat di Bab 3.

Kebenaran algoritma ini dibuktikan dengan asumsi bobot sisi berbeda [9, Bag. 7], [1, Bag. 11.6], yang dipenuhi oleh aturan pemutus seri pada butir 7 subbab 1.4. Tanpa urutan total atas sisi, sisi terpilih dapat membentuk siklus. Pada segitiga dengan tiga bobot sama, misalnya, ketiga simpul dapat memilih tiga sisi yang berbeda.

2.5 Penelitian Terkait

Tiga dari empat penelitian terbaru berikut mengembangkan varian algoritma MST (paralel, terdistribusi, dan aproksimasi), sedangkan satu menerapkan MST pada *clustering*.

Fallin *et al.* [3] mengembangkan ECL-MST, implementasi MST untuk GPU yang menyatukan algoritma Kruskal dan Borůvka dalam satu pendekatan paralelisasi. Pada GPU Titan V, ECL-MST dilaporkan rata-rata 4,6 kali lebih cepat daripada kode tercepat berikutnya. Pada sistem kedua (RTX 3080 Ti), delapan optimasi yang dievaluasi, bila digabung, membuat kodenya lebih dari 8 kali lebih cepat daripada versi tanpa optimasi tersebut.

Sanders dan Schimek [5] mengembangkan varian Borůvka dan Filter-Borůvka untuk memori terdistribusi. Algoritma mereka berskala hingga sedikitnya 65.536 inti dan hingga 800 kali lebih cepat daripada algoritma MST terdistribusi sebelumnya (nilai tertinggi pada graf grid yang berlokalitas tinggi). Mereka mencatat bahwa Borůvka dapat memproses seluruh sisi sebanyak logaritmik kali, dan Filter-Kruskal dalam banyak hal merupakan algoritma sekuensial praktis terbaik saat ini. Mereka juga menyatakan bahwa label simpul dapat dipakai untuk memutus seri secara konsisten sehingga MST menjadi unik, sejalan dengan aturan pemutus seri pada 1.4.

Almansoori *et al.* [6] mengusulkan algoritma pembangkitan MST aproksimasi untuk data berdimensi tinggi dan berukuran besar. Metode ini membangun graf k tetangga terdekat secara aproksimasi, membentuk MST awal darinya, lalu mengoptimalkannya. Mereka melaporkan keunggulan terhadap FISHDBC dalam waktu jalan dan galat relatif, sedangkan algoritma MST eksak lebih cepat pada data berdimensi rendah atau berukuran kecil.

Gagolewski *et al.* [4] menempatkan MST sebagai representasi data pada tugas *clustering* berdimensi rendah. Dari batas atas kesesuaian yang dihitung dengan algoritma *oracle*, mereka menyimpulkan bahwa metode berbasis MST berpotensi sangat kompetitif. Genie dan metode berbasis teori informasi sering mengungguli algoritma non-MST seperti *K-means*, *Gaussian mixture*, dan *spectral clustering*, dengan temuan terbatas pada *dataset* uji berukuran $n < 10.000$ dan dimensi $d \le 3$.

Laporan ini tidak mengusulkan varian baru. Laporan ini menganalisis secara formal dan mengukur secara sekuensial algoritma klasik Kruskal, Prim, dan Borůvka, yang menjadi acuan varian-varian tersebut, pada graf jarang dan padat, lalu membandingkan hasilnya dengan kurva kompleksitas teoretis.

BAB 3 ANALISIS ALGORITMA

3.1 Contoh Perhitungan Manual

Algoritma Kruskal dan Prim dijalankan langkah demi langkah pada dua graf kecil agar cara kerjanya dapat

diperiksa secara manual. Graf pertama, empat simpul dengan bobot berbeda, memperlihatkan penolakan sisi yang membentuk siklus. Graf kedua memuat sisi berbobot sama dan memperlihatkan peran aturan pemutus seri pada butir 7 subbab 1.4. Kedua graf sama dengan Kasus 1 dan Kasus 3 pada uji kecil di kode program, sehingga hasil perhitungan manual dapat dibandingkan langsung dengan keluaran program (Bab 4). Simpul diberi nomor tetap, bobot berupa bilangan bulat contoh yang sengaja tidak dikaitkan dengan data nyata, setiap sisi ditulis (u, v) dengan $u < v$, dan dalam antrean prioritas Prim ditulis sebagai tripel (w, u, v) sesuai kunci pembanding pada 2.2.3.

3.1.1 Graf Empat Simpul dengan Bobot Berbeda

Graf pertama memiliki $n = 4$ simpul dan $m = 5$ sisi, sehingga setiap *spanning tree* memiliki $n - 1 = 3$ sisi (lihat 1.2). Bobot sisinya adalah $w(0,1) = 1$, $w(1,2) = 2$, $w(0,2) = 3$, $w(2,3) = 4$, dan $w(1,3) = 5$.

Algoritma Kruskal. Karena seluruh bobot berbeda, urutan sisi menaik tunggal: (0, 1), (1, 2), (0, 2), (2, 3), (1, 3). Pada awalnya hutan terdiri atas empat pohon, yaitu $\{0\}$, $\{1\}$, $\{2\}$, dan $\{3\}$. Setiap sisi diperiksa menurut urutan itu dan pemeriksaan dihentikan setelah tiga sisi diterima (Tabel 3.1).

TABEL 3.1. JEJAK ALGORITMA KRUSKAL PADA GRAF EMPAT SIMPUL

Langkah	Sisi	w	Keputusan	Pohon sesudahnya	Total
1	(0, 1)	1	Terima	{0, 1}, {2}, {3}	1
2	(1, 2)	2	Terima	{0, 1, 2}, {3}	3
3	(0, 2)	3	Tolak (satu pohon)	{0, 1, 2}, {3}	3
4	(2, 3)	4	Terima	{0, 1, 2, 3}	7

Pada langkah 3, simpul 0 dan 2 sudah terhubung lewat simpul 1, sehingga sisi (0, 2) menutup siklus 0, 1, 2. Bobotnya, yaitu 3, lebih besar daripada kedua sisi lain pada siklus, sesuai dengan *cycle property* (Teorema 2.2). Setelah langkah 4 terkumpul $n - 1 = 3$ sisi, sehingga sisi (1, 3) tidak pernah diperiksa. Hasilnya adalah {(0, 1), (1, 2), (2, 3)} dengan total bobot 7.

Algoritma Prim. Pohon dimulai dari simpul 0, sehingga $S = \{0\}$, dan antrean berisi sisi yang meninggalkan simpul 0, yaitu (1, 0, 1) dan (3, 0, 2). Pada setiap langkah, tripel terkecil dikeluarkan. Jika kedua ujungnya sudah berada di S, tripel itu usang dan dibuang. Jika tidak, sisi diterima, ujung barunya masuk ke S, dan sisi dari ujung baru ke simpul di luar S dimasukkan ke antrean (Tabel 3.2).

TABEL 3.2. JEJAK ALGORITMA PRIM PADA GRAF EMPAT SIMPUL, SIMPUL AWAL 0

Langkah	Antrean sebelum pengambilan	Diambil	Keputusan	Total
1	(1, 0, 1), (3, 0, 2)	(1, 0, 1)	Terima, $S = \{0,1\}$	1
2	(2, 1, 2), (3, 0, 2), (5, 1, 3)	(2, 1, 2)	Terima, $S = \{0,1,2\}$	3
3	(3, 0, 2), (4, 2, 3), (5, 1, 3)	(3, 0, 2)	Buang, kedua ujung di S	3
4	(4, 2, 3), (5, 1, 3)	(4, 2, 3)	Terima, $S = \{0,1,2,3\}$	7

Pada langkah 3, tripel (3, 0, 2) sudah usang karena simpul 2 masuk lewat sisi (1, 2) pada langkah 2. Entri usang inilah yang dibiarkan di antrean (lihat 3.2.3). Setelah tiga sisi terkumpul, perulangan berhenti dan (5, 1, 3) tetap tersisa. Hasilnya {(0, 1), (1, 2), (2, 3)} dengan total bobot 7, sama dengan Kruskal. Graf ini memiliki delapan *spanning tree* (sepuluh kombinasi tiga sisi dikurangi dua kombinasi yang membentuk siklus) dengan total bobot 7 sampai 12, dan hanya satu yang bernilai 7. Jadi hasil keduanya memang MST, dan MST-nya unik sesuai Teorema 2.3.

3.1.2 Graf dengan Sisi Berbobot Sama

Graf kedua memiliki $n = 4$ simpul dan $m = 5$ sisi. Empat sisi (0, 1), (1, 2), (2, 3), dan (0, 3) membentuk siklus dan semuanya berbobot 2, sedangkan diagonal (1, 3) berbobot 5. Setiap *spanning tree* memiliki tiga sisi dan bobot sisi terkecil adalah 2, sehingga total bobot minimal 6. Membuang salah satu dari empat sisi pada siklus menghasilkan empat MST dengan total bobot sama. Dengan aturan pemutus seri pada butir 7 subbab 1.4 (2.2.3), urutan seluruh sisi menjadi tunggal, yaitu (2, 0, 1), (2, 0, 3), (2, 1, 2), (2, 2, 3), (5, 1, 3).

Algoritma Kruskal. Sisi diperiksa menurut urutan itu. Sisi (0, 1), (0, 3), dan (1, 2) diterima berturut-turut dengan total bobot 2, 4, dan 6, lalu pemeriksaan berhenti sehingga (2, 3) tidak diperiksa. Hasilnya {(0, 1), (0, 3), (1, 2)} dengan total bobot 6.

Algoritma Prim. Pohon dimulai dari simpul 0 (Tabel 3.3).

TABEL 3.3. JEJAK ALGORITMA PRIM PADA GRAF DENGAN SISI BERBOBOT SAMA, SIMPUL AWAL 0

Langkah	Antrean sebelum pengambilan	Diambil	Keputusan	Total
1	(2, 0, 1), (2, 0, 3)	(2, 0, 1)	Terima, $S = \{0,1\}$	2
2	(2, 0, 3), (2, 1, 2), (5, 1, 3)	(2, 0, 3)	Terima, $S = \{0,1,3\}$	4
3	(2, 1, 2), (2, 2, 3), (5, 1, 3)	(2, 1, 2)	Terima, $S = \{0,1,2,3\}$	6

Pada langkah 3, tripel (2, 1, 2) dan (2, 2, 3) berbobot sama, dan aturan pemutus seri memilih (2, 1, 2). Tanpa aturan itu, Prim dapat memilih (2, 2, 3) dan menghasilkan MST lain dengan total bobot yang sama. Hasilnya {(0, 1), (0, 3), (1, 2)} dengan total bobot 6, sama dengan Kruskal. Karena itu seluruh perbandingan keluaran pada laporan ini memakai aturan yang sama.

3.2 Pseudocode

Pseudocode disusun menurut kode program yang ditulis sendiri, sehingga setiap baris dapat ditelusuri ke satu baris atau satu blok pada kode. Graf $G = (V, E)$ memiliki n simpul bernomor 0 sampai $n - 1$ dan m sisi, dan setiap sisi ditulis (u, v, w). Kruskal dan Borůvka menerima graf sebagai daftar sisi, sedangkan Prim menerima daftar ketetanggaan *Adj*, yaitu Adj$[x]$ berisi pasangan (t, w) untuk setiap sisi yang menghubungkan x dengan tetangga t (butir 4 subbab 1.4).

Ketiga algoritma memakai kunci pembanding sisi yang sama, yaitu aturan pemutus seri pada butir 7 subbab 1.4. Untuk sisi $e = (u, v, w)$, kunci ini adalah tripel

$$\kappa(e) = \big(w, \min(u, v), \max(u, v)\big), \tag{5}$$

yang dibandingkan secara leksikografis. Karena graf tidak memiliki sisi ganda, tidak ada dua sisi dengan kunci yang sama, sehingga κ mengurutkan seluruh sisi secara total. Pengurutan pada Kruskal dan antrean prioritas pada Prim memakai alat bantu bawaan Python, yaitu `sorted` dan `heapq` (butir 3 subbab 1.4). Pada *pseudocode*, alat bantu itu ditulis sebagai "urutkan" dan operasi MASUKKAN serta KELUARKAN-TERKECIL, dan biayanya dibahas pada 3.4.

3.2.1 Disjoint Set

Kruskal memakai struktur *disjoint set* untuk menjawab apakah dua simpul sudah berada pada pohon yang sama (lihat 2.1.2), dan Borůvka memakai struktur yang sama untuk menentukan komponen setiap simpul. Struktur ini memakai *union by rank* dan kompresi lintasan [1, Bag. 11.4], dengan *path halving* sebagai varian kompresi yang dipilih.

PSEUDOCODE 3.1. DISJOINT SET

```
BUAT-HIMPUNAN(n)
1  untuk i ← 0 sampai n − 1 lakukan
2      induk[i] ← i ; peringkat[i] ← 0


CARI-AKAR(x)
1  selama induk[x] ≠ x lakukan
2      induk[x] ← induk[induk[x]]
3      x ← induk[x]
4  kembalikan x


GABUNG(a, b)
1  ra ← CARI-AKAR(a) ; rb ← CARI-AKAR(b)
2  jika ra = rb maka kembalikan SALAH
3  jika peringkat[ra] < peringkat[rb] maka tukar ra dan rb
4  induk[rb] ← ra
5  jika peringkat[ra] = peringkat[rb] maka
       peringkat[ra] ← peringkat[ra] + 1
6  kembalikan BENAR
```

GABUNG mengembalikan SALAH jika kedua simpul sudah berada pada himpunan yang sama. Nilai ini dipakai sebagai pendeteksi siklus oleh Kruskal dan sebagai penjaga oleh Borůvka.

3.2.2 Algoritma Kruskal

Kruskal mengurutkan seluruh sisi menurut κ, lalu memeriksanya satu per satu. Sisi diterima jika GABUNG berhasil dan ditolak jika kedua ujungnya sudah berada pada pohon yang sama. Perulangan dihentikan segera setelah $n - 1$ sisi terkumpul (lihat 3.1.1).

PSEUDOCODE 3.2. KRUSKAL

```
KRUSKAL(n, E)
1   E′ ← URUTKAN(E) menaik berdasarkan bobot w
2   himpunan ← BUAT-HIMPUNAN(n)
3   A ← ∅
4   total ← 0
5   untuk setiap sisi (u, v, w) pada E′ lakukan:
6       jika GABUNG(u, v) bernilai BENAR:
7           A ← A ∪ {(u, v, w)}
8           total ← total + w
9           jika |A| = n − 1 maka hentikan perulangan
10  jika |A| < n − 1 maka galat "Graf tidak terhubung"
11  kembalikan (A, total)
```

Jika graf tidak terhubung, sisi yang tersedia habis sebelum $n - 1$ sisi terkumpul dan baris 10 melaporkan galat. Pemeriksaan ini hanya pengaman, karena graf masukan diasumsikan terhubung (butir 1 subbab 1.4). Pada baris 1, sisi berbobot sama diurutkan menurut κ (butir 7 subbab 1.4), sehingga urutannya total.

3.2.3 Algoritma Prim

Prim tumbuh dari satu simpul awal s, dengan $s = 0$ pada implementasi ini. Rumusan berbasis simpul memerlukan operasi *decrease-key* (lihat 2.1.2), tetapi `heapq` tidak menyediakannya. Karena itu antrean prioritas Q menyimpan sisi, bukan simpul. Setiap sisi yang menghubungkan pohon dengan simpul di luar pohon dimasukkan ke Q, dan sisi yang ujung lainnya sudah berada di pohon dibuang saat dikeluarkan (baris 8), sehingga Q dapat memuat entri usang dan hingga $O(m)$ elemen. Elemen antrean adalah kunci κ dari sisi, sehingga urutan pengeluaran otomatis mengikuti aturan pemutus seri.

PSEUDOCODE 3.3. PRIM

```
BUAT-DAFTAR-TETANGGA(n, E)
1  untuk i ← 0 sampai n − 1 lakukan Adj[i] ← daftar kosong
2  untuk setiap (u, v, w) pada E lakukan
3      tambahkan (v, w) ke Adj[u] ; tambahkan (u, w) ke Adj[v]
4  kembalikan Adj


PRIM(n, Adj, s)
1   masuk ← SALAH untuk semua simpul
2   masuk[s] ← BENAR
3   A ← ∅
4   total ← 0
5   Q ← masukkan semua sisi dari s ke antrean prioritas
6   selama Q tidak kosong dan |A| < n − 1 lakukan:
7       (w, k, b) ← keluarkan sisi dengan bobot
            minimum dari Q
8       jika masuk[k] dan masuk[b] maka lewati
9       tujuan ← b jika tidak masuk[b] selain itu k
10      masuk[tujuan] ← BENAR
11      A ← A ∪ {(k, b, w)}
12      total ← total + w
13      masukkan ke Q sisi dari tujuan ke tetangga
            yang belum masuk
14  jika |A| < n − 1 maka galat "Graf tidak terhubung"
15  kembalikan (A, total)
```

Setiap elemen Q memiliki sedikitnya satu ujung di dalam pohon, sehingga pada baris 9 tepat satu ujung bernilai `masuk` dan ujung lainnya menjadi simpul baru. Perulangan berhenti ketika $n - 1$ sisi terkumpul atau Q kosong. Pada kasus kedua graf tidak terhubung dan baris 14 melaporkan galat. Elemen Q dibandingkan menurut κ, sehingga $k < b$ pada tripel (w, k, b) di baris 7. Pengukuran di Bab 4 melaporkan waktu Prim tanpa dan dengan pembuatan *Adj*.

3.2.4 Algoritma Borůvka

Borůvka bekerja dalam putaran. Pada setiap putaran, setiap komponen memilih sisi terkecil menurut κ di antara sisi yang keluar dari komponen itu (baris 7 sampai 9), dan baru setelah semua komponen memilih, seluruh pilihan dipasang (baris 11 sampai 15). Pemeriksaan komponen pada baris 8 memakai CARI-AKAR, yang hanya memendekkan lintasan dan tidak mengubah akar sebuah komponen. Sisi yang lebih murah pada baris 9 ditentukan menurut κ (butir 7 subbab 1.4).

PSEUDOCODE 3.4. BORŮVKA

```
BORŮVKA(n, E)
1   himpunan ← BUAT-HIMPUNAN(n)
2   A ← ∅
3   total ← 0
4   c ← n
5   selama c > 1 lakukan:
6       T ← ∅ (pencarian sisi termurah per komponen)
7       untuk setiap sisi (u, v, w) pada E lakukan:
8           jika u dan v dalam komponen yang sama maka lewati
9           simpan (u, v, w) ke T untuk komponen u dan v
                jika lebih murah
10      jika T kosong maka galat "Graf tidak terhubung"
11      untuk setiap sisi (u, v, w) pada T lakukan:
12          jika GABUNG(u, v) bernilai BENAR:
13              A ← A ∪ {(u, v, w)}
14              total ← total + w
15              c ← c − 1
16  kembalikan (A, total)
```

Satu sisi dapat dipilih oleh dua komponen sekaligus. Pada baris 12, GABUNG yang kedua kalinya mengembalikan SALAH, sehingga sisi itu tidak dipasang dua kali. Jika pada suatu putaran tidak ada sisi yang keluar dari komponen mana pun padahal $c > 1$, graf tidak terhubung dan baris 10 melaporkan galat.

3.3 Bukti Kebenaran

Kebenaran Kruskal dan Prim dibuktikan dengan invarian bahwa himpunan sisi A yang terkumpul selalu termuat dalam suatu MST, sehingga setiap sisi yang diterima adalah *safe edge* (lihat 2.1.2). Invarian itu dijaga oleh *cut property* (Teorema 2.1), yang buktinya dilengkapi lebih dulu pada 3.3.1. Seluruh pembuktian memakai graf $G = (V, E)$ tak

berarah, terhubung, dan berbobot real dengan $n = |V|$ dan $m = |E|$ (lihat 1.2).

3.3.1 Bukti Cut Property

Bukti Teorema 2.1. Misalkan T adalah MST yang memuat A, $(S, V \setminus S)$ adalah potongan yang menghormati A, dan $e = (u, v)$ adalah *light edge* pada potongan itu. Jika $e \in T$, maka $A \cup \{e\} \subseteq T$ dan pernyataan terbukti.

Jika $e \notin T$, karena T adalah *spanning tree*, T memuat tepat satu lintasan p dari u ke v. Kedua ujung e berada di sisi yang berlawanan pada potongan, sehingga p memuat sedikitnya satu sisi $f = (x, y)$ yang melintasi potongan. Karena potongan menghormati A, $f \notin A$. Menghapus f memutuskan satu-satunya lintasan antara u dan v di T, sehingga $T - \{f\}$ terdiri atas dua komponen, yang satu memuat u dan yang lain memuat v. Sisi e menghubungkan kedua komponen itu, sehingga $T' = (T - \{f\}) \cup \{e\}$ terhubung dan memiliki $n - 1$ sisi, yaitu sebuah *spanning tree*.

Karena e adalah *light edge* dan f melintasi potongan yang sama, $w(e) \leq w(f)$. Akibatnya $w(T') = w(T) - w(f) + w(e) \leq w(T)$. Karena T minimum, $w(T') = w(T)$, sehingga T' juga MST. Karena $f \notin A$, $A \subseteq T - \{f\}$, sehingga $A \cup \{e\} \subseteq T'$. □ [1, Lemma 11.1]

3.3.2 Kebenaran Algoritma Kruskal

Teorema 3.1 (Kebenaran Kruskal). Pada graf G yang tak berarah, terhubung, dan berbobot real, KRUSKAL (*Pseudocode* 3.2) mengembalikan sebuah MST dari G, dan galat pada baris 10 tidak pernah terjadi.

Bukti. Tiga fakta dipakai. Pertama, himpunan pada *disjoint set* selalu sama dengan komponen terhubung graf (V, A). Pada awalnya setiap simpul berdiri sendiri, dan A bertambah tepat ketika GABUNG mengembalikan BENAR (baris 6 sampai 8), yaitu ketika kedua ujung berada pada komponen yang berbeda. Dengan demikian GABUNG mengembalikan SALAH tepat ketika kedua ujung sudah berada pada komponen yang sama, dengan kebenaran struktur *disjoint set* sendiri diasumsikan [1, Bag. 11.4]. Kedua, karena A hanya bertambah, komponen hanya menyatu dan tidak pernah terpisah. Ketiga, sisi diperiksa menurut κ naik, sehingga bobot sisi yang diperiksa tidak pernah menurun.

Invarian: sebelum setiap sisi diperiksa pada baris 5, A termuat dalam suatu MST dari G.

Basis. Pada awalnya $A = \varnothing$, yang termuat dalam setiap MST, dan MST ada karena G terhubung (lihat 1.2).

Langkah. Misalkan $e = (u, v, w)$ adalah sisi yang sedang diperiksa. Jika e ditolak, A tidak berubah. Jika e diterima, misalkan C adalah komponen (V, A) yang memuat u, sehingga $v \notin C$. Potongan $(C, V \setminus C)$ menghormati A karena C adalah sebuah komponen, dan e melintasinya. Setiap sisi f yang sudah diperiksa sebelum e tidak melintasi potongan ini. Jika f diterima, $f \in A$ sehingga f tidak melintasi potongan yang menghormati A. Jika f ditolak, kedua ujungnya sudah berada pada komponen yang sama saat itu, dan karena komponen hanya menyatu, kedua ujungnya berada pada komponen yang sama sekarang. Jadi setiap sisi yang melintasi potongan belum diperiksa atau adalah e sendiri, sehingga bobotnya tidak kurang dari w. Dengan demikian e adalah *light edge* pada potongan yang menghormati A, dan menurut Teorema 2.1, $A \cup \{e\}$ termuat dalam suatu MST. Invarian terjaga.

Terminasi. Perulangan berhenti karena A mencapai $n - 1$ sisi (baris 9) atau karena seluruh sisi sudah diperiksa. Pada kasus kedua, andaikan (V, A) memiliki lebih dari satu komponen. Karena G terhubung, ada sisi $f \in E$ yang kedua ujungnya berada pada komponen berbeda. Sisi f sudah diperiksa, dan saat itu kedua ujungnya juga berada pada komponen berbeda karena komponen hanya menyatu, sehingga f diterima dan menjadi anggota A, yang bertentangan dengan kedua ujungnya berada pada komponen berbeda. Jadi (V, A) terhubung. Setiap sisi yang diterima menghubungkan dua komponen berbeda sehingga tidak pernah membentuk siklus, dan dengan demikian A adalah *spanning tree* dengan $n - 1$ sisi. Pada kedua kasus A memiliki $n - 1$ sisi, sehingga baris 10 tidak pernah melaporkan galat.

Kesimpulan. Setelah perulangan, A termuat dalam suatu MST T^*, dan $|A| = n - 1 = |T^*|$, sehingga $A = T^*$. □

3.3.3 Kebenaran Algoritma Prim

Teorema 3.2 (Kebenaran Prim). Pada graf G yang tak berarah, terhubung, dan berbobot real, untuk sembarang simpul awal s, PRIM (*Pseudocode* 3.3) mengembalikan sebuah MST dari G, dan galat pada baris 14 tidak pernah terjadi.

Bukti. Misalkan S adalah himpunan simpul dengan `masuk` bernilai BENAR. Invarian berikut berlaku pada awal setiap iterasi perulangan baris 6:

(a) (S, A) adalah sebuah pohon, yaitu A adalah himpunan sisi pohon dengan himpunan simpul S;

(b) A termuat dalam suatu MST dari G;

(c) kunci κ dari setiap sisi yang tepat satu ujungnya berada di S ada di Q, dan setiap elemen Q adalah kunci κ dari sebuah sisi yang sedikitnya satu ujungnya berada di S.

Basis. Setelah baris 1 sampai 4, $S = \{s\}$ dan $A = \varnothing$, sehingga (a) dan (b) berlaku. Setiap sisi yang tepat satu ujungnya di S adalah sisi yang bersisian dengan s, dan baris 5 memasukkan kuncinya ke Q. Seluruh elemen Q adalah kunci sisi yang bersisian dengan s, sehingga (c) berlaku.

Langkah. Misalkan (w, k, b) adalah elemen terkecil yang dikeluarkan pada baris 7, yaitu kunci κ dari sisi $e = (k, b)$.

Kasus 1: kedua ujung e berada di S (baris 8). Elemen dibuang dan S serta A tidak berubah, sehingga (a) dan (b) tetap berlaku. Sisi e tidak melintasi $(S, V \setminus S)$, sehingga kehilangan elemen ini tidak melanggar bagian pertama (c), dan bagian kedua (c) tetap berlaku untuk elemen yang tersisa.

Kasus 2: tepat satu ujung e berada di S. Menurut bagian kedua (c), setidaknya satu ujung berada di S, sehingga tepat satu ujungnya di S jika kasus 1 tidak terjadi. Misalkan *tujuan* adalah ujung yang tidak berada di S (baris 9). Potongan $(S, V \setminus S)$ menghormati A, karena menurut (a) kedua ujung setiap sisi A berada di S, dan e melintasinya. Setiap sisi f yang melintasi potongan memiliki kunci di Q menurut (c), dan $\kappa(e)$ adalah elemen terkecil Q, sehingga $\kappa(e) \leq \kappa(f)$ dan $w(e) \leq w(f)$. Jadi e adalah *light edge*, dan menurut Teorema 2.1 serta (b), $A \cup \{e\}$ termuat dalam suatu MST, sehingga (b) berlaku setelah baris 11. Karena tujuan $\notin S$ dihubungkan ke pohon oleh e, $(S \cup \{\text{tujuan}\}, A \cup \{e\})$ tetap berupa pohon, sehingga (a) berlaku. Untuk (c), sisi yang tepat satu ujungnya di $S \cup \{\text{tujuan}\}$ adalah sisi yang sebelumnya tepat satu ujungnya di S dan ujung lainnya bukan *tujuan*, yang kuncinya sudah ada di Q dan tidak ikut dikeluarkan, atau sisi dari *tujuan* ke simpul di luar $S \cup \{\text{tujuan}\}$, yang kuncinya dimasukkan pada baris 13. Setiap elemen Q sedikitnya memiliki satu ujung di $S \cup \{\text{tujuan}\}$.

Terminasi. Karena (S, A) adalah pohon, $|A| = |S| - 1$. Perulangan berhenti ketika $|A| = n - 1$ atau Q kosong. Andaikan Q kosong dan $S \neq V$. Karena G terhubung dan S tak kosong, ada sisi yang tepat satu ujungnya di S, dan kuncinya ada di Q menurut (c), yang bertentangan dengan Q kosong. Jadi pada kedua kasus $S = V$ dan $|A| = n - 1$, sehingga baris 14 tidak pernah melaporkan galat.

Kesimpulan. Menurut (b), A termuat dalam suatu MST T^*, dan $|A| = n - 1 = |T^*|$, sehingga $A = T^*$. □

3.3.4 Keunikan MST dan Keluaran Kruskal dan Prim pada Aturan Pemutus Seri

Teorema 3.1 dan 3.2 menjamin bahwa keluaran keduanya adalah sebuah MST, padahal pada bobot sama sebuah graf dapat memiliki beberapa MST (lihat 3.1.2). Keunikan MST pada bobot berbeda (Teorema 2.3) dibuktikan lebih dulu karena menjadi dasar teorema berikutnya.

Bukti Teorema 2.3. Buktinya memakai syarat yang lebih umum daripada bobot berbeda, yaitu setiap potongan G hanya memiliki satu *light edge*. Misalkan T_1 dan T_2 adalah MST dengan $e \in T_1$ tetapi $e \notin T_2$. Menghapus e dari T_1 memecah T_1 menjadi dua komponen, sehingga terbentuk sebuah potongan yang hanya dilintasi e dari sisi-sisi T_1. Setiap MST memuat sedikitnya satu *light edge* dari setiap potongan (*cut rule*; [9, Bag. 8]), sebab jika semua sisi MST yang melintasi potongan bukan *light edge*, penukaran seperti pada 3.3.1 dengan sebuah *light edge* menghasilkan *spanning tree* yang lebih ringan secara ketat. Jadi e, satu-satunya sisi T_1 pada potongan itu, adalah *light edge*-nya, dan satu-satunya menurut syarat tersebut. Karena T_2 juga MST, T_2 memuat *light edge* itu, yaitu e, yang bertentangan dengan $e \notin T_2$. Jadi $T_1 \subseteq T_2$, dan karena keduanya memiliki $|V| - 1$ sisi, $T_1 = T_2$. Bobot sisi yang berbeda membuat *light edge* pada setiap potongan menjadi tunggal, sehingga Teorema 2.3 terbukti. □ [9, Bag. 7 dan 8], [1, Latihan 11.5]

Teorema 3.3 (Keluaran sama). Misalkan $\rho(e) \in \{1, \ldots, m\}$ adalah peringkat sisi e menurut κ, dengan peringkat 1 untuk κ terkecil. Karena κ berbeda untuk setiap pasangan sisi (lihat 2.2.3), ρ berbeda untuk setiap sisi, sehingga graf G dengan bobot ρ memiliki tepat satu MST M menurut Teorema 2.3. Maka KRUSKAL dan PRIM pada G dengan bobot w, untuk sembarang simpul awal s, mengembalikan M, dan M adalah MST dari G menurut bobot w.

Bukti. Keputusan KRUSKAL pada baris 5 sampai 9 hanya bergantung pada urutan sisi menurut κ dan pada keadaan *disjoint set*, dan keputusan PRIM pada baris 6 sampai 13 hanya bergantung pada perbandingan elemen Q, yaitu perbandingan κ. Karena perbandingan κ identik dengan perbandingan ρ, jalannya kedua algoritma dengan bobot w identik dengan jalannya dengan bobot ρ. Menurut Teorema 3.1 dan 3.2 pada bobot ρ, keluaran keduanya adalah MST dari G menurut ρ, yaitu M yang unik. Menurut Teorema 3.1 pada bobot w, keluaran KRUSKAL juga MST dari G menurut w, sehingga M adalah MST menurut w. □

Pada graf 3.1.2, peringkat sisi adalah (0, 1) = 1, (0, 3) = 2, (1, 2) = 3, (2, 3) = 4, dan (1, 3) = 5, sehingga $M = \{(0,1), (0,3), (1,2)\}$, sama dengan keluaran kedua algoritma pada Tabel 3.3. Teorema ini menjadi dasar perbandingan himpunan sisi antar algoritma pada Bab 4. Borůvka tidak dibuktikan secara formal (butir 2 subbab 1.4), sehingga kesamaan keluarannya dengan M hanya diperiksa lewat uji kasus kecil dan graf acak.

3.4 Analisis Kompleksitas

Batas atas waktu dan ruang ketiga algoritma diturunkan dari *pseudocode* pada 3.2 dengan notasi O saja (2.3), dan berlaku untuk urutan bobot sisi apa pun pada kepadatan yang bersangkutan. Penurunan dilakukan baris demi baris agar setiap suku dapat ditelusuri ke satu blok pada *pseudocode*.

3.4.1 Asumsi, Alat Bantu, dan Disjoint Set

Graf G terhubung dan memiliki $n \geq 2$ simpul, sehingga $n - 1 \leq m < n^2$. Dua akibatnya dipakai berulang kali, yaitu $n = O(m)$ sehingga suku n terserap oleh suku m, dan $\log m < 2 \log n$ sehingga $O(\log m) = O(\log n)$. Operasi dasar, yaitu perbandingan, penugasan, dan akses larik, berbiaya $O(1)$, termasuk perbandingan dua kunci κ karena kunci berupa tripel. Alat bantu bawaan Python diperlakukan menurut biaya standar, yaitu `sorted` atas k elemen berbiaya $O(k \log k)$ pada kasus terburuk, serta `heapq.heappush` dan `heapq.heappop` pada antrean berisi k elemen masing-masing $O(\log k)$. Pengurutan bawaan Python bersifat adaptif sehingga dapat lebih cepat pada masukan yang hampir terurut. Karena itu laporan ini hanya mengklaim batas atas O, bukan Θ.

Teorema 3.4. Pada struktur *disjoint set Pseudocode* 3.1 yang berisi n elemen, BUAT-HIMPUNAN berbiaya $O(n)$, serta CARI-AKAR dan GABUNG masing-masing berbiaya $O(\log n)$ pada kasus terburuk.

Bukti. BUAT-HIMPUNAN menjalankan satu perulangan sebanyak n kali, sehingga berbiaya $O(n)$. Untuk dua operasi lainnya, dibuktikan dua sifat berikut.

(a) Peringkat simpul selalu lebih kecil daripada peringkat induknya. Pada awalnya setiap simpul adalah akar berperingkat 0. Pada GABUNG, baris 3 memastikan peringkat$[r_a] \geq$ peringkat$[r_b]$ sebelum r_b dipasang di bawah r_a, dan jika peringkat keduanya sama, baris 5 menaikkan peringkat$[r_a]$. Peringkat simpul yang bukan akar tidak pernah berubah, sedangkan peringkat induknya hanya bisa naik. Pada CARI-AKAR baris 2, induk$[x]$ diganti dengan leluhur yang peringkatnya lebih besar lagi. Jadi sifat (a) tetap berlaku setelah setiap operasi.

(b) Akar berperingkat r memiliki sedikitnya 2^r simpul pada pohonnya. Pernyataan ini benar pada awal ($r = 0$, satu simpul). Peringkat akar hanya naik pada baris 5 ketika dua akar berperingkat r digabung, sehingga pohon baru memuat sedikitnya $2^r + 2^r = 2^{r+1}$ simpul. Penggabungan dengan peringkat berbeda tidak mengubah peringkat akar dan hanya menambah simpul, dan pemendekan lintasan tidak mengeluarkan simpul dari pohonnya.

Dari (b), peringkat setiap akar paling besar $\log_2 n$. Dari (a), peringkat sepanjang lintasan menuju akar naik secara ketat, sehingga lintasan dari simpul mana pun memuat paling banyak $\log_2 n$ sisi. Jadi CARI-AKAR berbiaya $O(\log n)$, dan GABUNG, yang memanggil CARI-AKAR dua kali dan menambah langkah berbiaya $O(1)$, juga $O(\log n)$. □

3.4.2 Algoritma Kruskal

Teorema 3.5. *Pseudocode* 3.2 berjalan dalam waktu $O(m \log n)$.

Bukti. Baris 1 mengurutkan m sisi dengan biaya $O(m \log m) = O(m \log n)$. Baris 2 berbiaya $O(n)$. Baris 3 dan 4 berbiaya $O(1)$. Baris 5 sampai 9 memuat paling banyak m iterasi, masing-masing satu GABUNG berbiaya $O(\log n)$ (Teorema 3.4) dan pekerjaan $O(1)$, sehingga berbiaya $O(m \log n)$. Baris 10 dan 11 berbiaya $O(1)$. Jumlahnya $O(n + m \log n)$, dan karena $n = O(m)$, jumlah ini adalah $O(m \log n)$. □

Penghentian dini pada baris 9 tidak menurunkan batas ini. Pengurutan selalu dikerjakan lebih dulu pada seluruh sisi, dan perulangan tetap dapat memeriksa semua m sisi jika sisi terberat adalah satu-satunya jembatan graf.

3.4.3 Algoritma Prim

Pembuktian diawali dengan batas banyaknya masukan ke antrean, karena batas ini yang membedakan analisis Prim berbasis sisi dari rumusan berbasis simpul pada 2.1.2.

Lemma 3.1. Pada *Pseudocode* 3.3, setiap sisi dimasukkan ke Q paling banyak satu kali. Akibatnya jumlah MASUKKAN dan jumlah KELUARKAN-TERKECIL masing-masing paling banyak m, dan Q tidak pernah memuat lebih dari m elemen.

Bukti. Sisi (u,v) dimasukkan hanya pada baris 5 atau baris 13, yaitu ketika salah satu ujungnya, misalnya u, baru masuk pohon dan ujung lainnya v belum masuk pohon. Ketika v kelak masuk, baris 13 memeriksa tetangga v dan menemukan `masuk[u]` bernilai BENAR, sehingga sisi yang sama tidak dimasukkan lagi. Jadi setiap sisi dimasukkan paling banyak satu kali. Elemen hanya dapat dikeluarkan setelah dimasukkan, dan setiap elemen dikeluarkan paling banyak satu kali. □

Teorema 3.6. *Pseudocode* 3.3, termasuk BUAT-DAFTAR-TETANGGA, berjalan dalam waktu $O(m \log n)$.

Bukti. BUAT-DAFTAR-TETANGGA membuat n daftar kosong dan memproses setiap sisi dengan dua penambahan, sehingga berbiaya $O(n+m)$. Pada PRIM, baris 1 sampai 4 berbiaya $O(n)$. Setiap simpul masuk pohon paling banyak satu kali (baris 10), dan pada saat itu daftar tetangganya dipindai satu kali. Jumlah panjang seluruh daftar tetangga adalah $2m$, sehingga seluruh pemindaian berbiaya $O(m)$ di luar operasi antrean. Menurut Lemma 3.1, antrean menerima paling banyak m MASUKKAN dan melayani paling banyak m KELUARKAN-TERKECIL, dan setiap operasi berbiaya $O(\log m) = O(\log n)$. Pengeluaran elemen usang (baris 8) hanya berbiaya satu KELUARKAN-TERKECIL, sehingga sudah terhitung. Jumlah seluruhnya $O(n + m + m \log n) = O(m \log n)$. □

Batas ini berlaku untuk tumpukan biner dengan antrean berisi sisi (3.2.3). Rumusan berbasis simpul dengan *decrease-key* pada *Fibonacci heap* menghasilkan $O(m + n \log n)$ (subbab 1.1), tetapi batas tersebut tidak berlaku untuk kode yang ditulis dalam laporan ini.

3.4.4 Algoritma Borůvka

Lemma 3.2. Pada graf terhubung, *Pseudocode* 3.4 berhenti setelah paling banyak $\lceil \log_2 n \rceil$ putaran.

Bukti. Misalkan sebuah putaran dimulai dengan $c \geq 2$ komponen. Karena graf terhubung, setiap komponen memiliki sedikitnya satu sisi yang keluar, sehingga T memuat satu sisi untuk setiap komponen. Sisi yang dipilih komponen r menghubungkan r dengan komponen lain r'. Pada baris 11 sampai 15, GABUNG berhasil menyatukan keduanya, atau mengembalikan SALAH karena keduanya sudah disatukan lebih dulu pada putaran yang sama. Dalam kedua kasus, r dan r' berakhir pada komponen yang sama. Jadi setiap komponen pada akhir putaran memuat sedikitnya dua komponen awal putaran, sehingga c paling banyak menjadi $\lfloor c/2 \rfloor$. Setelah k putaran, jumlah komponen paling banyak $n/2^k$, yang bernilai 1 atau kurang untuk $k = \lceil \log_2 n \rceil$. □

Teorema 3.7. *Pseudocode* 3.4 berjalan dalam waktu $O(m \log^2 n)$.

Bukti. Baris 1 sampai 4 berbiaya $O(n)$. Bagian pemilihan setiap putaran (baris 7 sampai 9) memproses m sisi, masing-masing dengan dua CARI-AKAR berbiaya $O(\log n)$ menurut Teorema 3.4, sehingga berbiaya $O(m \log n)$. Bagian pemasangan (baris 11 sampai 15) memproses paling banyak n sisi, masing-masing satu GABUNG berbiaya $O(\log n)$, sehingga berbiaya $O(n \log n) = O(m \log n)$. Menurut Lemma 3.2 terdapat paling banyak $O(\log n)$ putaran, sehingga totalnya $O(m \log^2 n)$. □

Batas ini lebih besar satu faktor $\log n$ daripada $O(m \log n)$ pada pustaka (subbab 2.4). Rumusan pustaka menganggap komponen setiap simpul tersedia dalam waktu $O(1)$, sedangkan kode ini menentukan komponen dengan CARI-AKAR yang berbiaya $O(\log n)$. Batas pustaka dapat dicapai jika komponen disimpan sebagai label yang diperbarui pada setiap putaran, tetapi itu bukan kode yang diukur pada Bab 4. Karena itu $O(m \log^2 n)$ adalah batas atas yang dapat dibuktikan untuk kode ini, dan Bab 4 membandingkan kurva pengukuran dengan $O(m \log n)$ dan $O(m \log^2 n)$.

3.4.5 Kompleksitas Ruang

Teorema 3.8. Ruang yang dipakai ketiga algoritma, di luar masukan, adalah $O(n+m)$ untuk Kruskal dan Prim, dan $O(n)$ untuk Borůvka.

Bukti. Kruskal menyimpan salinan terurut E' sebanyak m elemen, struktur *disjoint set* dengan dua larik berukuran n, dan A dengan paling banyak $n-1$ elemen, sehingga ruangnya $O(n+m)$. Prim menyimpan Adj dengan $2m$ entri dan n daftar, larik `masuk` berukuran n, A, dan Q dengan paling banyak m elemen (Lemma 3.1), sehingga ruangnya $O(n+m)$. Borůvka menyimpan struktur *disjoint set* berukuran n, kamus T dengan paling banyak satu entri untuk setiap akar sehingga paling banyak n entri, dan A, sehingga ruang tambahannya $O(n)$. Daftar sisi masukan E tidak disalin. □

3.4.6 Ringkasan dan Substitusi ke Kasus Analisis

Hasil 3.4.2 sampai 3.4.5 dirangkum pada Tabel 3.4. Kasus analisis didefinisikan menurut kepadatan graf pada Tabel 2.1, sehingga batas pada Tabel 3.4 dihitung dengan mensubstitusikan nilai m ke dalamnya, dan hasilnya disajikan pada Tabel 3.5. Pada kasus rata-rata, $m = \Theta(n \log n)$ sehingga $\log m = \Theta(\log n)$ dan $m \log n = O(n \log^2 n)$. Substitusi ini adalah batas atas kepadatan menengah yang dipilih sebagai wakil (2.3), bukan nilai harapan waktu berjalan.

TABEL 3.4. RINGKASAN BATAS ATAS KOMPLEKSITAS (GRAF TERHUBUNG, $n \geq 2$)

Algoritma	Waktu	Ruang tambahan	Dasar
Kruskal	$O(m \log n)$	$O(n+m)$	Teorema 3.5, 3.8
Prim (tumpukan biner, antrean berisi sisi)	$O(m \log n)$	$O(n+m)$	Teorema 3.6, 3.8
Borůvka (seperti yang ditulis)	$O(m \log^2 n)$	$O(n)$	Teorema 3.7, 3.8

TABEL 3.5. SUBSTITUSI NILAI m MENURUT TABEL 2.1

Kasus	Nilai m	Waktu Kruskal dan Prim	Waktu Borůvka	Ruang Kruskal dan Prim	Ruang Borůvka
Terbaik	$n-1$	$O(n \log n)$	$O(n \log^2 n)$	$O(n)$	$O(n)$
Rata-rata	$\Theta(n \log n)$	$O(n \log^2 n)$	$O(n \log^3 n)$	$O(n \log n)$	$O(n)$
Terburuk	$\Theta(n^2)$	$O(n^2 \log n)$	$O(n^2 \log^2 n)$	$O(n^2)$	$O(n)$

Kruskal dan Prim memiliki batas atas yang sama pada ketiga kasus. Notasi O tidak membedakan keduanya, dan selisihnya baru tampak pada faktor konstanta yang diukur di Bab 4 (lihat 3.5).

TABEL 3.6. PERBANDINGAN KRUSKAL, PRIM, DAN BORŮVKA MENURUT IMPLEMENTASI DALAM LAPORAN INI

Aspek	Kruskal	Prim	Borůvka
Struktur yang ditumbuhkan	Hutan	Satu pohon dari simpul awal	Hutan, semua komponen tumbuh serentak per putaran
Sisi yang dipilih	Sisi terkecil menurut κ yang menghubungkan dua pohon berbeda	Sisi terkecil menurut κ yang meninggalkan pohon	Sisi terkecil menurut κ yang keluar dari setiap komponen
Struktur data	*Disjoint set* dan pengurutan seluruh sisi	Antrean prioritas berisi sisi (`heapq`)	*Disjoint set* dan kamus pilihan per komponen
Representasi graf	Daftar sisi	Daftar ketetanggaan	Daftar sisi
Waktu dan ruang tambahan	$O(m \log n)$ dan $O(n+m)$	$O(m \log n)$ dan $O(n+m)$	$O(m \log^2 n)$ dan $O(n)$
Bukti kebenaran	Formal (Teorema 3.1)	Formal (Teorema 3.2)	Tidak formal (butir 2 subbab 1.4)

Dua fakta tentang jumlah pekerjaan per sisi menunjukkan mengapa notasi O tidak memisahkan Kruskal dari Prim. Pertama, Prim memasukkan tepat m entri ke antrean. Setiap sisi dimasukkan paling banyak satu kali (Lemma 3.1), dan pada graf terhubung ujung sisi yang masuk pohon lebih dulu selalu memasukkannya, sedangkan jumlah KELUARKAN-TERKECIL berada di antara $n-1$ dan m. Kedua, Kruskal memeriksa sisi sampai sisi terberat MST. Perulangan berhenti pada penerimaan sisi ke $(n-1)$, sehingga jumlah sisi yang diperiksa berada di antara $n-1$ dan m, sedangkan pengurutan tetap mencakup seluruh m sisi.

Akibatnya, pada graf yang mana pun, Kruskal dan Prim sama-sama memproses sisi dalam jumlah yang berorde m dan batas atasnya sama. Selisih waktu di antara keduanya hanya dapat berasal dari faktor konstanta, yaitu biaya satu GABUNG dibandingkan biaya satu MASUKKAN dan KELUARKAN-TERKECIL, serta biaya pembuatan daftar ketetanggaan pada Prim. Faktor konstanta tidak dapat ditentukan dari analisis asimtotik dan diukur pada Bab 4.

3.5.2 Kelebihan, Keterbatasan, dan Kesesuaian

Kruskal. Kelebihannya adalah masukan berupa daftar sisi langsung dapat diproses tanpa membangun struktur graf lain, dan seluruh keputusan berada pada satu urutan sisi. Keterbatasannya, seluruh sisi harus tersedia dan diurutkan lebih dulu, salinan terurut membuat ruang tambahannya $O(n+m)$, dan penghentian dini tidak mengurangi biaya pengurutan (3.4.2). Algoritma ini cocok bila graf sudah berupa daftar sisi, misalnya daftar jalur kabel beserta biayanya, dan bila graf cukup jarang.

Prim. Kelebihannya adalah pohon tumbuh dari satu simpul dan sisi dipilih dari batas pohon, sehingga alami bila graf sudah tersedia sebagai daftar ketetanggaan. Keterbatasannya berasal dari pilihan implementasi. Antrean menyimpan sisi dan memuat entri usang (3.2.3), sehingga ruangnya mencapai $O(m)$ dan batas waktunya $O(m \log n)$, bukan batas rumusan berbasis simpul (3.4.3). Jika graf berupa daftar sisi, daftar ketetanggaan harus dibangun lebih dulu dengan biaya $O(n+m)$.

Borůvka. Kelebihannya adalah pemilihan sisi tiap komponen pada satu putaran saling bebas sehingga secara konsep mudah diparalelkan [1, Bag. 11.6], meski berada di luar batasan laporan ini, dan ruang tambahannya paling kecil, yaitu $O(n)$. Keterbatasan pada implementasi ini ada dua. Kode tidak melakukan kontraksi, sehingga setiap putaran memindai seluruh m sisi termasuk sisi yang kedua ujungnya sudah berada dalam satu komponen. Komponen juga ditentukan dengan CARI-AKAR, sehingga batas yang dapat dibuktikan adalah $O(m \log^2 n)$, satu faktor $\log n$ di atas batas pustaka (3.4.4). Algoritma ini dipilih sebagai pembanding karena strateginya berbeda, bukan karena diharapkan lebih cepat pada kode ini.

Kesesuaian untuk studi kasus. Studi kasus pada Bab 4 adalah jaringan kabel antar lima gedung, yaitu graf dengan $n = 5$ dan m paling banyak 10. Pada ukuran sekecil itu perbedaan orde pertumbuhan tidak berarti, dan pilihan di antara ketiganya ditentukan oleh bentuk data. Daftar jalur beserta biayanya sudah berbentuk daftar sisi, sehingga Kruskal paling langsung dipakai. Ada pula keterbatasan yang berasal dari modelnya. MST meminimalkan total biaya, tetapi hasilnya berupa pohon sehingga putusnya satu jalur memutus jaringan, MST tidak memperhitungkan syarat lain seperti cadangan jalur atau kondisi medan, dan jalur berbiaya negatif yang membentuk siklus tidak dipilih. Jika syarat itu penting, masalahnya bukan lagi MST murni.

3.5.3 Alternatif

Untuk Prim, rumusan berbasis simpul dengan *Fibonacci heap* memberi batas yang lebih kecil (3.4.3; [8], [1, Bag. 11.2]), tetapi tidak dapat dibangun langsung dari `heapq`. Filter-Kruskal [8] adalah varian Kruskal yang tidak dibahas (butir 2 subbab 1.4). Untuk graf sangat besar, tersedia implementasi paralel dan terdistribusi [3], [5] yang berada di luar batasan laporan ini. Untuk graf kecil seperti studi kasus, tidak ada alasan memakai alternatif tersebut.

3.5.4 Prediksi Teoretis untuk Bab 4

Pengukuran pada Bab 4 hanya mencakup graf jarang, padat, dan jarang berbobot kembar (butir 5 subbab 1.4), dan hanya waktu eksekusi (butir 6), sehingga ruang hanya dianalisis secara teoretis. Karena O adalah batas atas, prediksi pada Tabel 3.7 dirumuskan sebagai hal yang dapat gagal, dan cara membacanya ditetapkan sebelum data diambil. Hasilnya diperiksa dengan membagi waktu terukur $T(n)$ oleh suku batas, misalnya $T(n)/(m \log n)$. Nilai yang hampir konstan terhadap n berarti batas itu ketat untuk kode ini, nilai yang menurun berarti batas itu longgar, dan nilai yang naik berarti batas itu salah atau ada pengaruh lain yang belum dihitung.

TABEL 3.7. PREDIKSI TEORETIS DAN CARA PENGUJIANNYA PADA BAB 4

No	Pertanyaan	Prediksi	Pemeriksaan
1	Pertumbuhan Kruskal dan Prim	T tidak tumbuh lebih cepat daripada $m \log n$ (Teorema 3.5 dan 3.6)	$T/(m \log n)$ terhadap n
2	Pertumbuhan Borůvka	T tidak tumbuh lebih cepat daripada $m \log^2 n$, dan dapat berada di antara $m \log n$ dan $m \log^2 n$ (Teorema 3.7)	$T/(m \log n)$ dan $T/(m \log^2 n)$
3	Mana yang lebih cepat, Kruskal atau Prim	Tidak ada prediksi dari analisis asimtotik. Osipov *et al.* [8] melaporkan Kruskal baik hingga sekitar $8n$ sisi pada C++, yang belum tentu berlaku pada Python	Bandingkan keduanya pada graf jarang dan padat. Dua kepadatan tidak cukup untuk menetapkan titik silang
4	Biaya daftar ketetanggaan	Prim dengan pembuatan *Adj* lebih lambat daripada tanpa pembuatan *Adj* sebesar suku $O(n + m)$	Ukur Prim tanpa dan dengan pembuatan *Adj* (3.2.3)
5	Pengaruh bobot kembar	Batas waktu tidak berubah, dan total bobot ketiga algoritma sama	Bandingkan waktu pada graf jarang berbobot kembar, total bobot pada setiap ulangan, dan himpunan sisi pada graf kecil berbobot kembar
6	Kesamaan keluaran	Total bobot ketiga algoritma sama pada graf eksperimen, dan himpunan sisi sama pada graf kecil (Teorema 3.3 untuk Kruskal dan Prim, tanpa bukti untuk Borůvka)	Bandingkan total bobot pada setiap ulangan dan himpunan sisi pada kasus uji kecil. Himpunan sisi pada graf eksperimen tidak dibandingkan

Jika pengukuran tidak sesuai dengan salah satu prediksi, ketidaksesuaian itu dilaporkan dan dibahas pada Bab 4, bukan disesuaikan.

BAB 4 IMPLEMENTASI DAN EKSPERIMEN

4.1 Studi Kasus: Jaringan Kabel antar Gedung Fakultas Teknik

Studi kasus memodelkan perencanaan jaringan kabel data yang menghubungkan lima gedung di Fakultas Teknik Universitas Hasanuddin, kampus Gowa, yaitu Arsitektur, Elektro, Geologi, Industri, dan Sipil. Seluruh gedung harus saling terhubung, langsung atau lewat gedung lain, dan fakultas ingin total biaya bersih pemasangan sekecil mungkin. Setiap gedung menjadi satu simpul ($n = 5$), setiap jalur kabel kandidat menjadi satu sisi ($m = 6$), dan biaya bersih pemasangan jalur dalam juta rupiah menjadi bobot sisi. Persoalannya adalah memilih himpunan jalur yang menghubungkan kelima gedung dengan total biaya bersih terkecil, yaitu MST pada graf tak berarah berbobot.

Dari sepuluh pasangan gedung, hanya enam yang menjadi jalur kandidat, sedangkan empat pasangan lainnya diasumsikan tidak layak dibangun karena terhalang bangunan atau jalan. Pihak universitas dalam skenario ini mendukung sebagian jalur, sehingga biaya bersih tiap jalur memiliki tiga makna. Biaya positif berarti fakultas memakai dana sendiri sebesar nilai tersebut. Biaya nol berarti seluruh biaya ditanggung universitas tanpa insentif bagi fakultas. Biaya negatif berarti seluruh biaya ditanggung universitas dan fakultas menerima insentif sebesar nilai mutlaknya. Bobot nol dan negatif sah karena bobot sisi adalah bilangan real (butir 1 subbab 1.4) dan algoritma hanya memakai urutan bobot (Teorema 2.1 dan 2.2). Seluruh biaya adalah nilai ilustrasi karangan, bukan data keuangan sesungguhnya, dan dipakai untuk memperlihatkan bahwa ketiga algoritma bekerja pada persoalan yang bermakna, bukan untuk menyatakan rancangan jaringan yang optimal bagi kampus.

TABEL 4.1. JALUR KABEL KANDIDAT DAN BIAYA BERSIHNYA (NILAI ILUSTRASI)

Gedung A	Gedung B	Biaya bersih (juta rupiah)
Arsitektur	Sipil	−5
Sipil	Elektro	−2
Elektro	Geologi	0
Sipil	Industri	15
Arsitektur	Elektro	18
Industri	Geologi	25

Seluruh biaya pada Tabel 4.1 berbeda, sehingga MST tunggal dan komponen pemutus seri pada kunci κ (persamaan (5)) tidak menentukan urutan sisi. Kruskal mengurutkan sisi secara menaik lalu menerima Arsitektur dan Sipil (−5), Sipil dan Elektro (−2), Elektro dan Geologi (0), serta Sipil dan Industri (15). Sisi terakhir menghubungkan dua komponen berbeda, yaitu {Arsitektur, Sipil, Elektro, Geologi} dan {Industri}. Setelah empat sisi ($n - 1$) diterima, pemeriksaan berhenti, dan sisi Arsitektur dan Elektro (18) serta Industri dan Geologi (25) tidak diperiksa. Keluaran Kruskal, Prim, dan Borůvka sama dengan perhitungan manual ini, yaitu empat sisi dengan total biaya bersih 8 juta rupiah ($-5 - 2 + 0 + 15$). Artinya, fakultas memakai dana sendiri Rp15 juta untuk jalur Sipil dan Industri dan menerima insentif Rp7 juta dari dua jalur lainnya. Program juga membandingkan himpunan sisi ketiganya secara langsung, dan hasilnya sama. Hal ini sesuai dengan keunikan MST pada graf ini.

Ada satu batasan makna pada bobot negatif. MST harus berupa pohon, sehingga jalur berbiaya negatif yang membentuk siklus tidak dipilih dan insentifnya tidak ikut dihitung. Hasil ini bukan solusi untuk masalah membangun jalur sebanyak mungkin demi insentif terbesar, yang merupakan masalah lain. Dengan hanya lima simpul, studi kasus ini menunjukkan kebenaran keluaran dan kegunaan MST, tetapi tidak dapat dipakai menilai efisiensi, yang dibahas pada eksperimen sintetis di subbab berikutnya.

4.2 Lingkungan Uji dan Implementasi

Seluruh pengukuran dijalankan pada satu sesi Google Colab versi gratis (*free tier*), yaitu mesin virtual dengan sumber daya yang tidak dijamin tetap. Spesifikasinya dicatat oleh program pada awal sesi (Tabel 4.2).

TABEL 4.2. LINGKUNGAN UJI

Komponen	Keterangan
Platform	Google Colab versi gratis (mesin virtual)
Prosesor	AMD EPYC 7B12, 2 CPU
Sistem operasi	Linux 6.6.122+
Python	3.13.16
Pustaka	pandas 2.2.3, numpy 2.1.3
Seed pengacak	2026
Pengukur waktu	`time.perf_counter()`
Eksekusi	sekuensial

Waktu diambil dengan `time.perf_counter()` hanya di sekitar pemanggilan algoritma, setelah `gc.collect()`, dan pengumpul sampah tetap aktif selama pengukuran. Ketiga algoritma dijalankan pada graf yang sama. Kruskal dan Borůvka menerima daftar sisi, sedangkan Prim menerima daftar ketetanggaan (butir 4 subbab 1.4), sehingga waktu

Prim dicatat dua kali, yaitu waktu inti tanpa pembuatan daftar ketetanggaan dan waktu yang ditambah konversi daftar sisi menjadi daftar ketetanggaan.

Ketiga algoritma ditulis dalam Python dan berjalan sekuensial (butir 3 subbab 1.4). Kode tersimpan pada notebook eksperimen di repositori, yaitu `CODING/EKSPERIMEN/TUGAS_MST_LOKAL.ipynb`. Kasus uji kecil dan studi kasus memakai berkas terpisah, yaitu `CODING/UJI_KECIL/UJI_KECIL.py` dan `CODING/STUDI_KASUS/STUDI_KASUS.py` (subbab 4.3). Pengurutan memakai `sorted` pada Kruskal dan antrean prioritas memakai `heapq` pada Prim, keduanya hanya sebagai alat bantu. Struktur *disjoint set* ditulis sendiri pada kelas `HimpunanTerpisah` dengan *union by rank* dan *path halving* (*Pseudocode* 3.1), dan Borůvka memakai kelas yang sama dari notebook yang sama. Kode 4.1 dan 4.2 mengutip bagian inti Kruskal dan Prim dengan komentar dihilangkan dan baris panjang dipatahkan agar muat. Kode Borůvka (*Pseudocode* 3.4) tersedia di repositori.

KODE 4.1. KRUSKAL (*PSEUDOCODE* 3.2)

```
sisi_terurut = sorted(
    daftar_sisi,
    key=lambda sisi: (sisi[2],
                      min(sisi[0], sisi[1]),
                      max(sisi[0], sisi[1])))
himpunan = HimpunanTerpisah(jumlah_simpul)
for simpul_a, simpul_b, bobot in sisi_terurut:
    if himpunan.gabung(simpul_a, simpul_b):
        sisi_mst.append((simpul_a, simpul_b, bobot))
        total_bobot += bobot
        if len(sisi_mst) == jumlah_simpul - 1:
            break
```

Kunci pengurutan adalah tripel κ pada persamaan (5). Sisi diterima hanya jika `gabung` mengembalikan `True`, yaitu jika kedua simpul belum berada pada himpunan yang sama, dan perulangan berhenti segera setelah $n-1$ sisi terkumpul. Jika jumlah sisi kurang dari $n-1$, fungsi melempar `ValueError` karena graf tidak terhubung (tidak ditampilkan pada kutipan).

KODE 4.2. PRIM (*PSEUDOCODE* 3.3)

```
while antrean and len(sisi_mst) < jumlah_simpul - 1:
    bobot, kecil, besar = heapq.heappop(antrean)
    if sudah_masuk[kecil] and sudah_masuk[besar]:
        continue
    asal, tujuan = ((kecil, besar) if sudah_masuk[kecil]
                    else (besar, kecil))
    sudah_masuk[tujuan] = True
    sisi_mst.append((asal, tujuan, bobot))
    total_bobot += bobot
    for tetangga, bobot_baru in daftar_tetangga[tujuan]:
        if not sudah_masuk[tetangga]:
            heapq.heappush(antrean, (
                bobot_baru,
                min(tujuan, tetangga),
                max(tujuan, tetangga)))
```

Antrean berisi sisi dalam bentuk tripel (bobot, simpul kecil, simpul besar), sehingga urutannya memakai kunci κ yang sama dengan Kruskal. Sisi termurah diambil dari antrean. Jika kedua ujungnya sudah berada di pohon, sisi itu usang dan dilewati (3.2.3). Jika tidak, ujung yang belum masuk menjadi simpul baru, lalu semua sisi dari simpul itu ke simpul di luar pohon dimasukkan ke antrean.

4.3 Verifikasi Kebenaran

Kebenaran kode diperiksa sebelum waktu dicatat, karena waktu dari kode yang salah tidak bermakna. Pemeriksaan ini adalah bukti empiris yang menguatkan. Bukti formal Kruskal dan Prim ada di Bab 3, sedangkan Borůvka tidak dibuktikan secara formal (butir 2 subbab 1.4). Lima pemeriksaan dilakukan.

Pertama, empat kasus uji kecil (data pada repositori, Lampiran A), yaitu tiga graf terhubung dengan total 7, 5, dan 6 serta satu graf tak terhubung yang harus menghasilkan `ValueError`. Ketiga algoritma lolos pada keempat kasus, dan total 7 dan 6 sama dengan perhitungan manual pada 3.1. Kasus 2 (lima simpul) memuat sisi berbobot 0 dan -2 sehingga bobot nol dan negatif ikut diuji, dan Kasus 3 berbobot kembar membandingkan himpunan sisi. Kedua, total bobot ketiga algoritma dibandingkan dengan hasil `networkx` pada 300 graf acak kecil, dengan `networkx` hanya sebagai pembanding (butir 3 subbab 1.4). Ketiga, *seed* yang sama menghasilkan graf yang sama dan *seed* berbeda menghasilkan graf lain, dan pada ukuran terkecil tiap skenario graf terhubung, tanpa sisi ganda, dan tanpa *loop*. Keempat, pada setiap ulangan pengukuran total bobot ketiga algoritma dibandingkan dan program berhenti jika berbeda. Kelima, pada studi kasus 4.1 program membandingkan himpunan sisi ketiga algoritma secara langsung.

Batasnya, pada graf eksperimen yang dibandingkan hanya total bobot, yang belum menjamin himpunan sisi yang sama. Satu keterbatasan lain menyangkut kode yang diukur. Pengukuran waktu pada subbab 4.5 dijalankan di Colab, sedangkan keempat kasus uji kecil dan studi kasus 4.1 dijalankan terpisah dari berkas `UJI_KECIL/UJI_KECIL.py` dan `STUDI_KASUS/STUDI_KASUS.py` di repositori. Kedua berkas itu berdiri sendiri dan lebih sederhana daripada kode di notebook. Struktur *disjoint set* di dalamnya hanya memakai kompresi lintasan tanpa *union by rank*, dan Prim menyusun daftar ketetanggaan di dalam programnya sendiri. Program studi kasus mengurutkan sisi hanya menurut bobot, yang aman karena seluruh bobot pada Tabel 4.1 berbeda. Karena itu kasus uji kecil dan studi kasus memeriksa kebenaran keluaran pada kode yang berdiri sendiri itu, bukan pada kode notebook yang diukur waktunya.

4.4 Rancangan Eksperimen

Pengukuran memakai tiga skenario graf (Tabel 4.3). Graf padat diberi ukuran lebih kecil karena jumlah sisinya tumbuh sebesar $\Theta(n^2)$ (butir 5 subbab 1.4). Dengan $m = n(n-1)/4$, yaitu separuh jumlah sisi maksimum, graf padat berukuran $n = 1000$ sudah memiliki sekitar $2{,}5 \times 10^5$ sisi, sehingga jumlah sisi pada kedua kepadatan berada pada rentang yang sebanding. Skenario ketiga memakai bentuk graf yang sama dengan skenario pertama, tetapi bobotnya hanya bilangan bulat 1 sampai 5 sehingga banyak sisi berbobot sama.

TABEL 4.3. SKENARIO PENGUKURAN

Skenario	Jumlah sisi m	Ukuran n	Bobot
jarang	$3n$	4000, 8000, 16000, 32000, 64000	bulat acak 1 sampai 10^6
padat	$n(n-1)/4$	200, 400, 600, 800, 1000	bulat acak 1 sampai 10^6
jarang_kembar	$3n$	4000, 8000, 16000, 32000, 64000	bulat acak 1 sampai 5

Graf dibangkitkan dengan `random.Random` ber-*seed* 2026. Pembangkit membentuk pohon acak terlebih dahulu, dengan setiap simpul dihubungkan ke satu simpul bernomor lebih kecil, sehingga graf pasti terhubung. Sisi acak lalu ditambahkan tanpa sisi ganda dan tanpa *loop* sampai jumlah sisi terpenuhi. Untuk setiap pasangan skenario dan ukuran, satu graf dibangkitkan. Satu putaran pemanasan dijalankan lebih dahulu dan tidak dicatat, lalu pengukuran diulang lima kali pada graf yang sama (butir 6 subbab 1.4). Karena graf yang sama dipakai pada kelima ulangan, simpangan baku hanya mengukur gangguan waktu pada mesin, bukan variasi antar graf. Hasil dilaporkan sebagai rata-rata dan simpangan baku sampel (pembagi $n-1$).

4.5 Hasil

Hasil pengukuran disajikan per skenario pada Tabel 4.4 sampai 4.6 sebagai rata-rata ± simpangan baku dari lima ulangan, dalam milidetik. Grafik waktu terhadap jumlah sisi beserta kurva teoretis ada pada Gambar 4.1 sampai 4.3, dan ringkasan rasio terhadap suku teoretis ada pada Tabel 4.7.

TABEL 4.4. WAKTU EKSEKUSI SKENARIO JARANG ($m = 3n$), RATA-RATA ± SIMPANGAN BAKU DALAM MS

n	m	Kruskal	Prim	Prim (+konversi)	Borůvka
4000	12000	14,41 ± 1,05	17,37 ± 0,62	24,39 ± 1,04	74,31 ± 2,41
8000	24000	35,43 ± 2,67	41,60 ± 2,36	55,88 ± 3,14	190,53 ± 5,42
16000	48000	111,68 ± 39,75	107,43 ± 22,25	144,62 ± 26,59	498,81 ± 204,16
32000	96000	214,93 ± 23,50	287,20 ± 42,33	364,16 ± 54,74	968,72 ± 63,24
64000	192000	518,64 ± 61,71	660,55 ± 45,82	972,07 ± 81,34	2445,55 ± 222,78

TABEL 4.5. WAKTU EKSEKUSI SKENARIO PADAT ($m = n(n-1)/4$), RATA-RATA ± SIMPANGAN BAKU DALAM MS

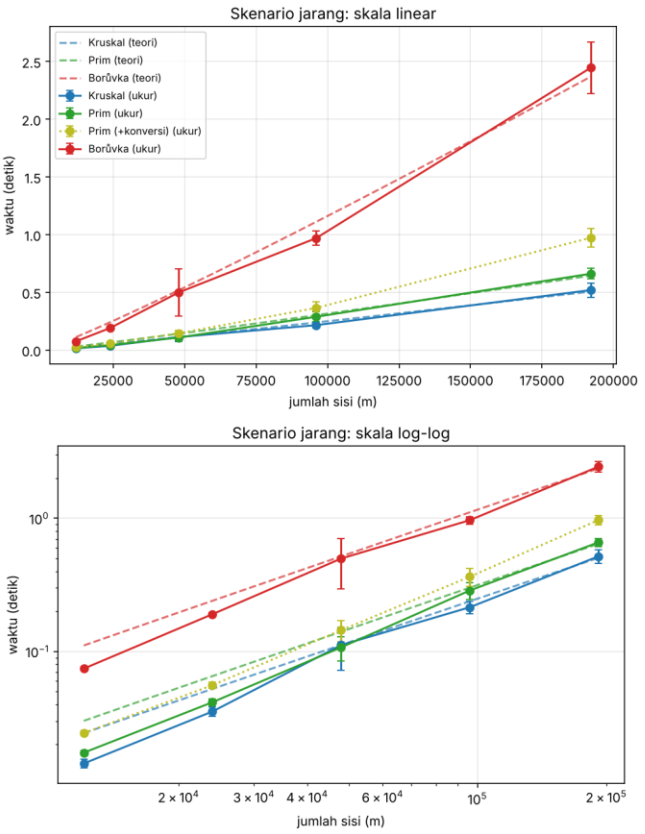
n	m	Kruskal	Prim	Prim (+konversi)	Borůvka
200	9950	6,91 ± 0,36	7,32 ± 0,76	11,67 ± 0,84	36,25 ± 1,50
400	39900	47,73 ± 10,48	38,56 ± 7,59	59,24 ± 8,68	247,49 ± 40,75
600	89850	138,17 ± 7,02	96,39 ± 8,72	144,51 ± 8,73	518,47 ± 3,95
800	159800	272,74 ± 12,49	192,68 ± 24,21	285,04 ± 23,90	1227,78 ± 180,97
1000	249750	494,40 ± 72,36	308,00 ± 18,05	462,61 ± 33,18	1828,52 ± 44,64

TABEL 4.6. WAKTU EKSEKUSI SKENARIO JARANG_KEMBAR ($m = 3n$, BOBOT 1 SAMPAI 5), RATA-RATA ± SIMPANGAN BAKU DALAM MS

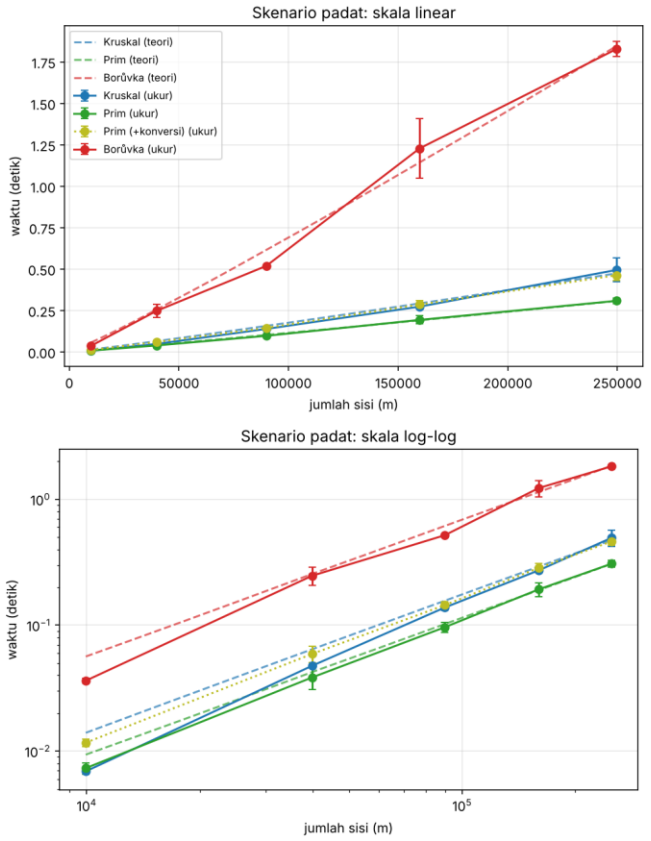
n	m	Kruskal	Prim	Prim (+konversi)	Borůvka
4000	12000	17,93 ± 3,35	19,05 ± 1,51	26,13 ± 1,77	73,38 ± 5,19
8000	24000	47,39 ± 11,42	52,03 ± 12,86	68,67 ± 15,29	190,26 ± 27,19
16000	48000	92,20 ± 9,85	96,13 ± 10,84	126,25 ± 11,57	386,68 ± 22,08
32000	96000	222,10 ± 14,80	240,43 ± 33,65	309,97 ± 31,03	903,79 ± 117,78
64000	192000	593,21 ± 96,25	647,42 ± 99,29	935,13 ± 123,85	1984,15 ± 47,14

TABEL 4.7. RASIO WAKTU TERHADAP SUKU BATAS PADA UKURAN TERBESAR (UKURAN TERKECIL = 1,00), EKSPONEN EMPIRIS α, DAN EKSPONEN LOKAL α DARI SUKU BATAS

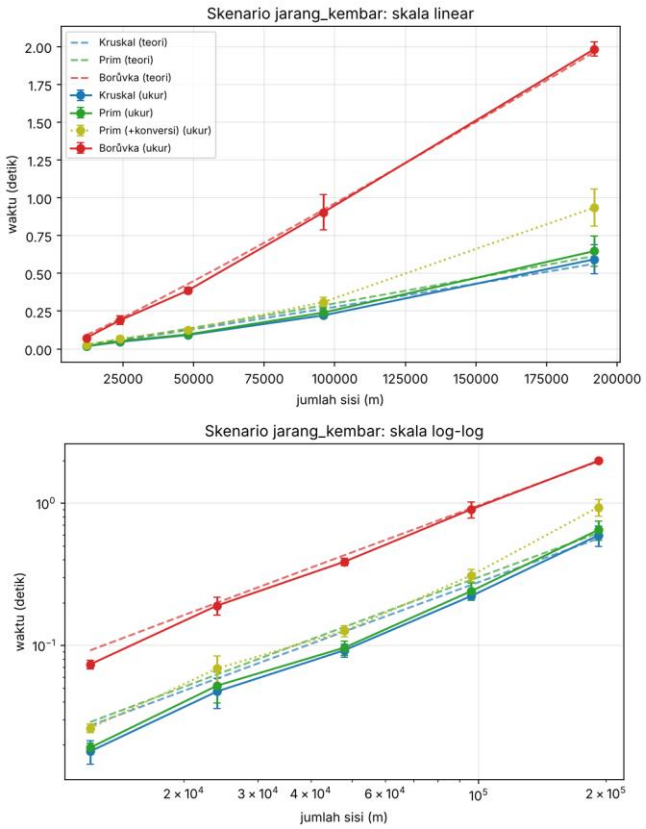
Skenario	Algoritma	Batas	Rasio	α empiris	α lokal
jarang	Kruskal	$m \log n$	1,69	1,292	1,104
jarang	Prim	$m \log n$	1,78	1,312	1,104
jarang	Borůvka	$m \log^2 n$	1,16	1,260	1,208
padat	Kruskal	$m \log n$	2,19	1,325	1,082
padat	Prim	$m \log n$	1,29	1,160	1,082
padat	Borůvka	$m \log^2 n$	1,18	1,217	1,165
jarang_kembar	Kruskal	$m \log n$	1,55	1,262	1,104
jarang_kembar	Prim	$m \log n$	1,59	1,272	1,104
jarang_kembar	Borůvka	$m \log^2 n$	0,95	1,189	1,208



Gambar 4.1. Waktu eksekusi terhadap jumlah sisi pada skenario jarang. Atas: skala linear. Bawah: skala log-log. Batang galat adalah simpangan baku, dan garis putus-putus adalah kurva teoretis dengan konstanta yang dicocokkan dengan kuadrat terkecil melalui titik asal.



Gambar 4.2. Waktu eksekusi terhadap jumlah sisi pada skenario padat, dengan keterangan yang sama dengan Gambar 4.1.



Gambar 4.3. Waktu eksekusi terhadap jumlah sisi pada skenario jarang_kembar, dengan keterangan yang sama dengan Gambar 4.1.

Kurva teoretis pada gambar memakai suku $m \log_2 m$ untuk Kruskal dan $m \log_2 n$ untuk Prim dan Borůvka. Kurva Prim (+konversi) tidak memiliki kurva teoretis sendiri. Rasio pada Tabel 4.7 dihitung dari rata-rata pada Tabel 4.4 sampai 4.6 dengan suku $m \log n$ (Kruskal dan Prim) dan $m \log^2 n$ (Borůvka), berlogaritma basis 2, agar sesuai dengan cara pemeriksaan pada Tabel 3.7, sehingga rasio Kruskal dapat berbeda dari keluaran notebook yang memakai $m \log_2 m$. Eksponen α dihitung dari ukuran terkecil dan terbesar, dan eksponen lokal batas dihitung dengan cara yang sama dari suku batasnya.

Data menunjukkan hal berikut. Pada skenario jarang, Kruskal memiliki rata-rata terendah pada empat ukuran, dan Prim terendah pada $n = 16000$ (107,43 dibandingkan 111,68 ms). Pada skenario jarang_kembar Kruskal terendah pada kelima ukuran. Pada skenario padat Prim tanpa konversi terendah pada empat ukuran ($n = 400$ sampai 1000), dan Kruskal terendah pada $n = 200$ (6,91 dibandingkan 7,32 ms). Borůvka paling lambat pada semua ukuran, yaitu 3,34 sampai 4,72 kali waktu Kruskal pada ukuran terbesar. Pembuatan daftar ketetanggaan menambah waktu Prim sebesar 47,2% (jarang), 50,2% (padat), dan 44,4% (jarang_kembar) pada ukuran terbesar. Simpangan baku berkisar dari 0,8% sampai 40,9% dari rata-rata, tertinggi pada Borůvka skenario jarang dengan $n = 16000$.

4.6 Pembahasan

Sesuai ketentuan di 3.5.4, ketidaksesuaian prediksi pada Tabel 3.7 dilaporkan apa adanya dan tidak disesuaikan.

4.6.1 Pertumbuhan Kruskal dan Prim (Prediksi 1)

Tidak terpenuhi dalam bentuk yang ditetapkan di 3.5.4. Jika batas $m \log n$ ketat, $T/(m \log n)$ hampir konstan, dan jika longgar, nilainya turun. Pada Tabel 4.7, nilainya pada ukuran terbesar berada di atas nilai ukuran terkecil pada keenam kombinasi, yaitu 1,69, 2,19, dan 1,55 kali untuk Kruskal serta 1,78, 1,29, dan 1,59 kali untuk Prim. Kenaikan terkecil (Prim, skenario padat) masih lebih besar daripada gangguan pengukuran, karena simpangan baku Prim pada skenario itu hanya 10,4% dari rata-rata pada $n = 200$ dan 5,9% pada $n = 1000$. Keenam eksponen empiris (1,160 sampai 1,325) juga lebih besar daripada eksponen lokal $m \log n$ (1,082 sampai 1,104). Artinya, pada data ini waktu Kruskal dan Prim tumbuh lebih cepat daripada $m \log n$.

Hasil ini tidak membantah Teorema 3.5 dan 3.6, karena keduanya adalah batas atas asimtotik, sedangkan data hanya mencakup rentang terbatas (faktor 16 pada m untuk skenario jarang dan jarang_kembar, dan 25,1 untuk skenario padat). Yang ditunjukkan data adalah bahwa model $T = c \cdot m \log n$ dengan konstanta tetap tidak cocok untuk kode ini pada rentang tersebut, sehingga ada pengaruh yang belum dimodelkan. Penyebabnya tidak diuji. Dugaan seperti pengaruh memori dan *cache* pada struktur data yang membesar, atau biaya alokasi dan *garbage collector* Python, masuk akal tetapi tidak dibedakan oleh pengukuran mana pun di laporan ini. Satu petunjuk tidak langsung adalah biaya pembuatan *Adj* per elemen $(n + m)$, yaitu selisih Prim (+konversi) dan Prim dibagi $n + m$. Pada ukuran terbesar biayanya lebih tinggi daripada pada ukuran terkecil di ketiga skenario, padahal operasi $O(n + m)$ seharusnya berbiaya per elemen tetap. Ini tetap petunjuk, bukan penjelasan yang terbukti, karena dihitung dari selisih dua rata-rata yang masing-masing memiliki gangguan.

4.6.2 Pertumbuhan Borůvka (Prediksi 2)

Terpenuhi sebagian. Rasio Borůvka terhadap $m \log^2 n$ pada ukuran terbesar adalah 1,16 (jarang), 1,18 (padat), dan 0,95 (jarang_kembar). Eksponen empirisnya melampaui eksponen lokal $m \log^2 n$ sebesar +0,052 pada skenario jarang dan padat, dan tidak melampauinya (−0,019) pada jarang_kembar, sehingga hanya satu dari tiga skenario yang tidak melampaui batas. Rasio terhadap $m \log^2 n$ lebih datar daripada terhadap $m \log n$ (1,54, 1,54, dan 1,27 pada ukuran terbesar), yang sejalan dengan Teorema 3.7 bahwa batasnya lebih besar daripada $m \log n$.

Hasil ini tidak dibaca sebagai bukti ketat. Pada dua dari tiga skenario waktu tumbuh sedikit lebih cepat daripada $m \log^2 n$. Titik $n = 16000$ pada skenario jarang memiliki simpangan baku sekitar 41% dari rata-rata, sehingga rasio di titik itu tidak ditafsirkan. Kruskal dan Prim, yang batasnya lebih rendah, juga tumbuh lebih cepat daripada batasnya (4.6.1), sehingga rasio yang tidak melampaui 1 pada jarang_kembar dapat berasal dari pengaruh yang sama yang menaikkan ketiganya, bukan dari ketatnya batas.

4.6.3 Kruskal dan Prim (Prediksi 3)

Tabel 3.7 tidak memuat prediksi arah untuk perbandingan ini, sehingga hasilnya dilaporkan saja. Kruskal terendah pada skenario jarang (empat dari lima ukuran) dan jarang_kembar (lima dari lima), dan Prim tanpa konversi terendah pada skenario padat (empat dari lima ukuran). Arah ini sejalan dengan laporan Osipov *et al.* [8] bahwa Kruskal baik pada graf yang tidak terlalu padat, tetapi kondisinya berbeda karena laporan itu memakai C++, sedangkan laporan ini memakai Python dengan `heapq` dan `sorted`.

Besar selisih perlu dibaca bersama simpangan baku. Pada skenario padat, selisih rata-rata melebihi simpangan baku kedua algoritma pada $n = 600$, 800, dan 1000. Pada skenario jarang, selisih melebihi simpangan baku kedua algoritma pada empat ukuran, kecuali $n = 16000$ yang selisihnya 4,25 ms jauh di bawah simpangan baku keduanya. Pada skenario jarang_kembar tidak ada ukuran yang selisihnya melebihi

kedua simpangan baku, sehingga urutannya tidak dapat dipastikan. Hasil ini juga tidak menetapkan titik silang, karena m/n pada skenario padat berkisar dari 49,75 sampai 249,75 sedangkan pada skenario jarang tetap 3, dan data tidak memuat kepadatan di antaranya. Keunggulan Prim pada skenario padat juga bergantung pada *Adj* yang sudah tersedia. Jika pembuatan *Adj* dihitung, Prim (+konversi) lebih lambat daripada Kruskal pada $n = 200$ sampai 800 dan lebih cepat hanya pada $n = 1000$ dengan selisih 31,79 ms yang lebih kecil daripada simpangan baku keduanya. Borůvka paling lambat pada semua ukuran, sejalan dengan faktor log tambahan pada Teorema 3.7 dan dengan pemindaian seluruh sisi pada setiap putaran, tetapi pembagian pengaruhnya tidak diukur.

4.6.4 Biaya Daftar Ketetanggaan (Prediksi 4)

Terpenuhi dalam arah dan tidak dalam besaran konstan. Prim (+konversi) lebih lambat daripada Prim tanpa konversi pada seluruh 15 ukuran. Biaya per elemen yang lebih tinggi pada ukuran terbesar (4.6.1) menunjukkan bahwa pada rentang ini suku $O(n + m)$ tidak berperilaku seperti biaya linear dengan konstanta tetap. Kolom "tanpa konversi" menyiratkan graf sudah tersedia sebagai *Adj*, sedangkan Kruskal dan Borůvka bekerja langsung pada daftar sisi, sehingga perbandingan yang adil bergantung pada bentuk data yang diterima program.

4.6.5 Pengaruh Bobot Kembar dan Kesamaan Keluaran (Prediksi 5 dan 6)

Prediksi 5 terpenuhi. Rasio waktu skenario jarang_kembar terhadap jarang pada n dan m yang sama berkisar dari 0,83 sampai 1,34 untuk Kruskal, 0,84 sampai 1,25 untuk Prim, dan 0,78 sampai 1,00 untuk Borůvka. Rasio itu tidak naik terhadap n, sehingga data tidak menunjukkan bobot kembar memperlambat pertumbuhan waktu. Arah selisihnya berbeda antar algoritma dan penyebabnya tidak diuji, sehingga selisih konstanta tidak ditafsirkan sebagai pengaruh bobot. Tidak adanya perlambatan sejalan dengan rancangan, karena kunci κ mengurutkan seluruh sisi secara total dan perbandingan κ tidak lebih mahal ketika bobotnya sama.

Prediksi 6 terpenuhi pada cakupan yang dirumuskan di Tabel 3.7. Total bobot ketiga algoritma sama pada setiap ulangan di 15 graf eksperimen, dan himpunan sisi dibandingkan pada kasus uji kecil dan studi kasus 4.1. Kesamaan himpunan sisi pada graf besar tidak diperiksa dan tidak diklaim. Untuk Kruskal dan Prim, kesamaan himpunan sisi dijamin oleh Teorema 3.3. Untuk Borůvka tidak ada bukti formal (butir 2 subbab 1.4), sehingga kesamaan total bobot dan himpunan sisi pada graf kecil adalah bukti empiris dan bukan pengganti bukti.

4.6.6 Ancaman terhadap Validitas

Hasil di atas berlaku dengan batas berikut.

1. **Gangguan waktu besar.** Simpangan baku mencapai sekitar 41% dari rata-rata (Borůvka, skenario jarang, $n = 16000$), dan 7 dari 60 kombinasi algoritma dan ukuran memiliki simpangan baku sekurang-kurangnya 20% dari rata-rata. Selisih yang lebih kecil daripada simpangan baku tidak ditafsirkan.
2. **Sumber daya Colab dan satu graf per ukuran.** Colab versi gratis tidak menjamin kecepatan mesin tetap selama sesi, dan setiap ukuran diwakili satu graf (*seed* 2026) sehingga simpangan baku hanya mengukur gangguan waktu, bukan variasi antar graf.
3. **Urutan eksekusi tetap.** Dalam setiap ulangan urutannya selalu pembuatan *Adj*, Kruskal, Prim, lalu Borůvka. Pengaruh urutan, misalnya keadaan memori atau *garbage collector*, tidak diacak dan tidak dapat dipisahkan dari perbedaan antar algoritma.
4. **Rentang ukuran sempit.** n terbesar 64000 pada graf jarang dan 1000 pada graf padat, jauh di bawah 10^6 pada contoh di slide ketentuan. Lima ukuran hanya mencakup faktor 16 sampai 25 pada m, sehingga eksponen empiris bersifat lokal, dan pada skenario padat n dan kepadatan berubah bersamaan.
5. **Cakupan terbatas.** Hanya dua kepadatan dan satu varian bobot kembar yang diuji, hanya dalam Python, dan hanya waktu yang diukur.

Rasio waktu terhadap teori pada ukuran terbesar masih di atas nilai ukuran terkecil untuk Kruskal dan Prim, sehingga pernyataan tentang pertumbuhan hanya berlaku pada rentang yang diukur dan tidak boleh diekstrapolasi ke ukuran yang lebih besar tanpa pengukuran tambahan.

BAB 5 KESIMPULAN

5.1 Kesimpulan

Kruskal menumbuhkan hutan dengan memeriksa sisi menurut urutan bobot, sedangkan Prim menumbuhkan satu pohon dari simpul awal dengan sisi termurah yang keluar dari pohon (3.1 dan 3.2). Keduanya terbukti benar, yaitu selalu menghasilkan *minimum spanning tree* (Teorema 3.1 dan 3.2), dan dengan aturan pemutus seri yang sama keduanya menghasilkan MST yang sama (Teorema 3.3).

Waktu Kruskal dan Prim sama-sama $O(m \log n)$, sedangkan Borůvka pada kode ini $O(m \log^2 n)$. Ruang tambahannya $O(n + m)$ untuk Kruskal dan Prim, dan $O(n)$ untuk Borůvka. Dengan kasus dibagi menurut kepadatan graf (2.3), batas Kruskal dan Prim adalah $O(n \log n)$ pada kasus terbaik, $O(n \log^2 n)$ pada kasus rata-rata, dan $O(n^2 \log n)$ pada kasus terburuk. Teori tidak membedakan Kruskal dan Prim, sehingga pembedanya dilihat dari pengukuran.

Ketiga algoritma diimplementasikan dalam Python, lolos empat kasus uji kecil, memberi total bobot yang sama dengan `networkx` pada 300 graf acak kecil, dan memberi hasil yang sama pada studi kasus jaringan kabel lima gedung Fakultas Teknik Universitas Hasanuddin, yaitu total biaya bersih 8 juta rupiah (biaya ilustrasi, memuat bobot 0 dan negatif).

Pengukuran pada graf jarang, padat, dan jarang berbobot kembar menunjukkan bahwa Kruskal memiliki rata-rata waktu terendah pada graf jarang (empat dari lima ukuran) dan graf jarang berbobot kembar (kelima ukuran), dan Prim terendah pada graf padat (empat dari lima ukuran) jika daftar ketetanggaan sudah tersedia. Borůvka paling lambat pada semua ukuran, yaitu 3,34 sampai 4,72 kali waktu Kruskal pada ukuran terbesar. Selisih Kruskal dan Prim sering masih berada di dalam simpangan baku, sehingga tidak ada satu algoritma yang terbaik pada semua kondisi. Terhadap kurva teoretis, waktu Kruskal dan Prim naik lebih cepat daripada $m \log n$ (rasio pada ukuran terbesar 1,29 sampai 2,19 kali nilai ukuran terkecil). Ini tidak membantah batas atas asimtotik karena rentang yang diukur terbatas, dan penyebabnya tidak diuji.

5.2 Kelebihan, Keterbatasan, dan Kesesuaian

Kruskal sederhana dan langsung memproses daftar sisi, tetapi seluruh sisi harus diurutkan lebih dulu. Prim alami dipakai pada daftar ketetanggaan, tetapi `heapq` tidak menyediakan *decrease-key*, sehingga batasnya $O(m \log n)$,

bukan $O(m + n \log n)$. Borůvka hemat ruang, tetapi kode ini memindai seluruh sisi pada setiap putaran dan kebenarannya tidak dibuktikan formal. Pada studi kasus lima gedung, ketiganya sesuai dan menghasilkan himpunan sisi yang sama, sehingga pilihan di antaranya ditentukan oleh bentuk data.

Hasil ini terbatas pada rentang ukuran yang jauh di bawah 10^6, satu graf per ukuran pada Colab versi gratis, dua kepadatan saja sehingga titik silang Kruskal dan Prim tidak ditetapkan, dan studi kasus berbiaya ilustrasi yang tidak memperhitungkan cadangan jalur.

5.3 Peluang Pengembangan

Pengembangan yang dapat dilakukan adalah memperlebar rentang ukuran dan kepadatan dengan beberapa graf per ukuran untuk menguji penyebab pertumbuhan yang lebih cepat daripada $m \log n$, mengganti antrean Prim dengan struktur yang mendukung *decrease-key* [8], [1], menambahkan kontraksi graf pada Borůvka dan menguji Filter-Kruskal [8], mengimplementasikan ketiganya dalam bahasa selain Python atau versi paralel [3], [5], serta mengganti jarak ilustrasi dengan hasil pengukuran lapangan.

DAFTAR PUSTAKA

[1] P. Sanders, K. Mehlhorn, M. Dietzfelbinger, and R. Dementiev, *Sequential and Parallel Algorithms and Data Structures: The Basic Toolbox*. Cham, Switzerland: Springer, 2019, doi: 10.1007/978-3-030-25209-0.

[2] R. C. Prim, "Shortest connection networks and some generalizations," *Bell Syst. Tech. J.*, vol. 36, no. 6, pp. 1389-1401, Nov. 1957, doi: 10.1002/j.1538-7305.1957.tb01515.x.

[3] A. Fallin, A. Gonzalez, J. Seo, and M. Burtscher, "A high-performance MST implementation for GPUs," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC '23)*, Denver, CO, USA, Nov. 2023, doi: 10.1145/3581784.3607093.

[4] M. Gagolewski, A. Cena, M. Bartoszuk, and Ł. Brzozowski, "Clustering with minimum spanning trees: How good can it be?," *J. Classif.*, vol. 42, no. 1, pp. 90-112, 2025, doi: 10.1007/s00357-024-09483-1.

[5] P. Sanders and M. Schimek, "Engineering massively parallel MST algorithms," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, St. Petersburg, FL, USA, May 2023, pp. 691-701, doi: 10.1109/IPDPS54959.2023.00075.

[6] M. K. M. Almansoori, A. Meszaros, and M. Telek, "Fast and memory-efficient approximate minimum spanning tree generation for large datasets," *Arab. J. Sci. Eng.*, vol. 50, no. 2, pp. 1233-1246, 2025, doi: 10.1007/s13369-024-08974-y.

[7] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," *Proc. Amer. Math. Soc.*, vol. 7, no. 1, pp. 48-50, 1956, doi: 10.1090/S0002-9939-1956-0078686-7.

[8] V. Osipov, P. Sanders, and J. Singler, "The Filter-Kruskal minimum spanning tree algorithm," in *Proc. Workshop Algorithm Eng. Exp. (ALENEX)*, 2009, pp. 52-61, doi: 10.1137/1.9781611972894.5.

[9] J. Nešetřil, E. Milková, and H. Nešetřilová, "Otakar Borůvka on minimum spanning tree problem: Translation of both the 1926 papers, comments, history," *Discrete Math.*, vol. 233, no. 1-3, pp. 3-36, 2001, doi: 10.1016/S0012-365X(00)00224-7.

[10] MIT OpenCourseWare, "Lecture 12: Greedy algorithms and minimum spanning tree," *6.046J Design and Analysis of Algorithms*, Massachusetts Inst. Technol., Cambridge, MA, USA, Spring 2015. [Online]. Available: https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/4a7fdddff3bc419c70bb470106a1663a_MIT6_046JS15_lec12.pdf. Accessed: Oct. 9, 2026.

LAMPIRAN

Lampiran A. Tautan Repositori dan Laporan Penggunaan AI

Seluruh kode, data, dan hasil eksperimen laporan ini tersimpan pada repositori pribadi penulis berikut.

Tautan repositori:
https://github.com/LippyyDev/Tugas-Besar-Algoritma-Komputasi

Laporan penggunaan AI ada pada bagian Penggunaan AI di *README* repositori tersebut.

Tautan laporan penggunaan AI:
https://github.com/LippyyDev/Tugas-Besar-Algoritma-Komputasi#penggunaan-ai